FRE7241 Algorithmic Portfolio Management

Lecture#7, Fall 2025

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

December 9, 2025

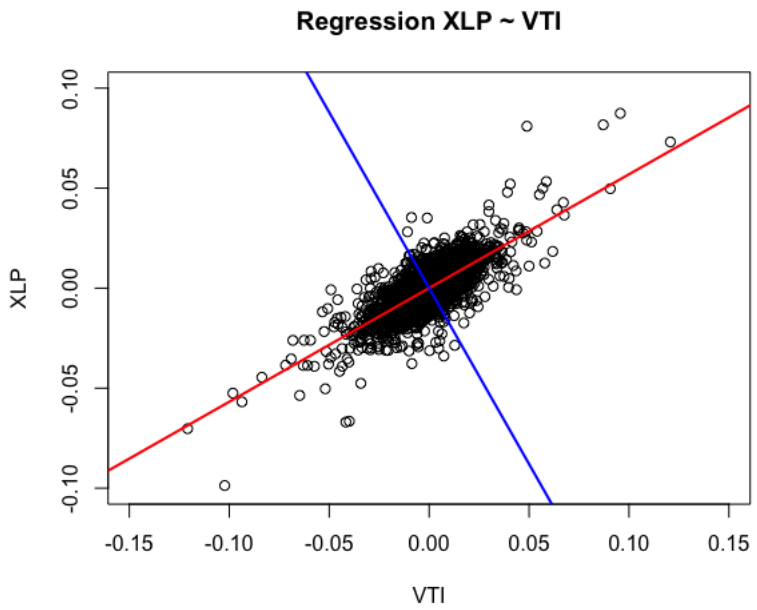
The *Alpha* and *Beta* of Stock Returns

The daily stock returns $r_i - r_f$ in excess of the risk-free rate r_f, can be decomposed into *systematic* returns $\beta(r_m - r_f)$ (where $r_m - r_f$ are the excess market returns) plus *idiosyncratic* returns $\alpha + \varepsilon_i$ (which are uncorrelated to the market returns):

$$r_i - r_f = \alpha + \beta(r_m - r_f) + \varepsilon_i$$

The *alpha* α are the abnormal returns in excess of the risk premium, and ε_i are the regression residuals with zero mean.

The *idiosyncratic* risk (equal to ε_i) is uncorrelated to the *systematic* risk, and can be reduced through portfolio diversification.




```
> # Perform regression using formula
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> raterf <- 0.03/252
> retp <- (retp - raterf)
> regmod <- lm(XLP ~ VTI, data=retp)
> regsum <- summary(regmod)
> # Get regression coefficients
> coef(regsum)
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 3.44e-05   7.97e-05   0.432    0.666
VTI         5.49e-01   6.58e-03  83.523    0.000
> # Get alpha and beta
> coef(regsum)[, 1]
(Intercept)         VTI
   3.44e-05    5.49e-01
```

```
> # Plot scatterplot of returns with aspect ratio 1
> plot(XLP ~ VTI, data=rutils::etfenv$returns, main="Regression XLP
+      xlim=c(-0.1, 0.1), ylim=c(-0.1, 0.1), pch=1, col="blue", asp=
> # Add regression line and perpendicular line
> abline(regmod, lwd=2, col="red")
> abline(a=0, b=-1/coef(regsum)[2, 1], lwd=2, col="blue")
```

The Statistical Significance of *Alpha* and *Beta*

The stock β is independent of the risk-free rate r_f:

$$\beta = \frac{\mathrm{Cov}(r_i, r_m)}{\mathrm{Var}(r_m)}$$

The *t*-statistic (*t*-value) is the ratio of the estimated value divided by its standard error.

The *p*-value is the probability of obtaining values exceeding the *t*-statistic, assuming the *null hypothesis* is true.

A small *p*-value means that the regression coefficients are very unlikely to be zero (given the data).

The *beta* β values of stock returns are very statistically significant, but the *alpha* α values are mostly not significant.

The *p*-value of the *Durbin-Watson* test is large, which indicates that the regression residuals are not autocorrelated.

In practice, the α, β, and the risk-free rate r_f, depend on the time interval of the data, so they're time dependent.

```
> # Get regression coefficients
> coef(regsum)
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 3.44e-05   7.97e-05   0.432    0.666
VTI         5.49e-01   6.58e-03  83.523    0.000
> # Calculate regression coefficients from scratch
> betac <- drop(cov(retp$XLP, retp$VTI)/var(retp$VTI))
> alphac <- drop(mean(retp$XLP) - betac*mean(retp$VTI))
> c(alphac, betac)
[1] 3.44e-05 5.49e-01
> # Calculate the residuals
> residuals <- (retp$XLP - (alphac + betac*retp$VTI))
> # Calculate the standard deviation of residuals
> nrows <- NROW(residuals)
> residsd <- sqrt(sum(residuals^2)/(nrows - 2))
> # Calculate the standard errors of beta and alpha
> sum2 <- sum((retp$VTI - mean(retp$VTI))^2)
> betasd <- residsd/sqrt(sum2)
> alphasd <- residsd*sqrt(1/nrows + mean(retp$VTI)^2/sum2)
> c(alphasd, betasd)
[1] 7.97e-05 6.58e-03
> # Perform the Durbin-Watson test of autocorrelation of residuals
> lmtest::dwtest(regmod)

 Durbin-Watson test

data:  regmod
DW = 2, p-value = 1
alternative hypothesis: true autocorrelation is greater than 0
```

The *Alpha* and *Beta* of ETF Returns

The *beta* β values of ETF returns are very statistically significant, but the *alpha* α values are mostly not significant.

Some of the ETFs with significant *alpha* α values are the bond ETFs *IEF* and *TLT* (which have performed very well), and the natural resource ETFs *USO* and *DBC* (which have performed very poorly).

```
> retp <- rutils::etfenv$returns
> symbolv <- colnames(retp)
> symbolv <- symbolv[symbolv != "VTI"]
> # Perform regressions and collect statistics
> betam <- sapply(symbolv, function(symbol) {
+ # Specify regression formula
+   formulav <- as.formula(paste(symbol, "~ VTI"))
+ # Perform regression
+   regmod <- lm(formulav, data=retp)
+ # Get regression summary
+   regsum <- summary(regmod)
+ # Collect regression statistics
+   with(regsum,
+     c(beta=coefficients[2, 1],
+ pbeta=coefficients[2, 4],
+ alpha=coefficients[1, 1],
+ palpha=coefficients[1, 4],
+ pdw=lmtest::dwtest(regmod)$p.value))
+ })  ##  end sapply
> betam <- t(betam)
> # Sort by palpha
> betam <- betam[order(betam[, "palpha"]), ]
```

```
> betam
        beta     pbeta     alpha   palpha      pdw
IEF  -0.1090 2.02e-125  1.85e-04 0.000592 5.57e-01
VXX  -2.8061  0.00e+00 -1.30e-03 0.001396 5.45e-01
GLD   0.0561  7.96e-06  3.90e-04 0.010561 7.97e-01
USO   0.6852 9.65e-157 -6.99e-04 0.023460 8.36e-02
VEU   0.9756  0.00e+00 -2.08e-04 0.024957 1.00e+00
TLT  -0.2330 7.69e-130  2.53e-04 0.025329 7.33e-01
XLF   1.2500  0.00e+00 -2.31e-04 0.052298 1.00e+00
IVE   0.9689  0.00e+00 -6.35e-05 0.184084 1.00e+00
VLUE  0.9672  0.00e+00 -1.05e-04 0.226198 9.81e-01
EEM   1.1730  0.00e+00 -1.59e-04 0.226526 1.00e+00
SVXY  2.1187 1.53e-239 -7.66e-04 0.235623 2.11e-07
AIEQ  1.1650 1.41e-200 -2.63e-04 0.237895 9.16e-01
IWD   0.9660  0.00e+00 -5.46e-05 0.241582 1.00e+00
VNQ   1.1400  0.00e+00 -1.86e-04 0.251250 1.00e+00
XLP   0.5493  0.00e+00  8.80e-05 0.269292 1.00e+00
USMV  0.7070  0.00e+00  6.44e-05 0.321409 8.64e-01
DBC   0.4002 7.27e-198 -1.40e-04 0.376844 9.35e-01
IVW   0.9826  0.00e+00  3.67e-05 0.385710 1.00e+00
QQQ   1.0966  0.00e+00  6.30e-05 0.456467 9.98e-01
XLE   1.0813  0.00e+00 -1.22e-04 0.457397 3.52e-01
XLV   0.7289  0.00e+00  6.00e-05 0.476510 8.29e-01
XLU   0.6357  0.00e+00  8.02e-05 0.502126 1.00e+00
XLB   1.0203  0.00e+00 -5.47e-05 0.580436 1.00e+00
VTV   0.9391  0.00e+00 -2.84e-05 0.603203 1.00e+00
QUAL  0.9682  0.00e+00  1.87e-05 0.641943 9.98e-01
IWF   1.0110  0.00e+00  2.88e-05 0.666126 1.00e+00
XLK   1.1099  0.00e+00  3.56e-05 0.676834 9.99e-01
MTUM  1.0108  0.00e+00  3.26e-05 0.731513 3.77e-02
XLY   1.0427  0.00e+00  2.43e-05 0.753965 1.00e+00
XLI   0.9967  0.00e+00 -1.93e-05 0.784320 1.00e+00
SPY   0.9860  0.00e+00 -5.39e-06 0.801208 1.00e+00
IWB   0.9830  0.00e+00 -2.04e-06 0.915229 1.00e+00
VYM   0.8579  0.00e+00 -5.08e-06 0.940619 1.00e+00
```

Capital Asset Pricing Model (*CAPM*)

The *CAPM* model states that the expected return for stock n: $\mathbb{E}[R_n]$ is proportional to its beta β_n times the expected excess return of the market $\mathbb{E}[R_m] - r_f$:

$$\mathbb{E}[R_n] = r_f + \beta_n(\mathbb{E}[R_m] - r_f)$$

The *CAPM* model states that if a stock has a higher beta then it's also expected to earn higher returns.

According to the *CAPM* model, assets are on average expected to earn only a *systematic* return proportional to their *systematic* risk.

The *CAPM* model is not a regression model.

The *CAPM* model depends on the choice of the risk-free rate r_f.

```
> library(PerformanceAnalytics)
> # Calculate XLP beta
> PerformanceAnalytics::CAPM.beta(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.549
> # Or
> retxlp <- na.omit(retp[, c("XLP", "VTI")])
> betac <- drop(cov(retxlp$XLP, retxlp$VTI)/var(retxlp$VTI))
> betac
[1] 0.549
> # Calculate XLP alpha
> PerformanceAnalytics::CAPM.alpha(Ra=retp$XLP, Rb=retp$VTI)
[1] 8.8e-05
> # Or
> mean(retp$XLP - betac*retp$VTI)
[1] NA
> # Calculate XLP bull beta
> PerformanceAnalytics::CAPM.beta.bull(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.564
> # Calculate XLP bear beta
> PerformanceAnalytics::CAPM.beta.bear(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.565
```

The Security Market Line for ETFs

The *Security Market Line* (SML) represents the linear relationship between expected stock returns and their *systematic* risk β.

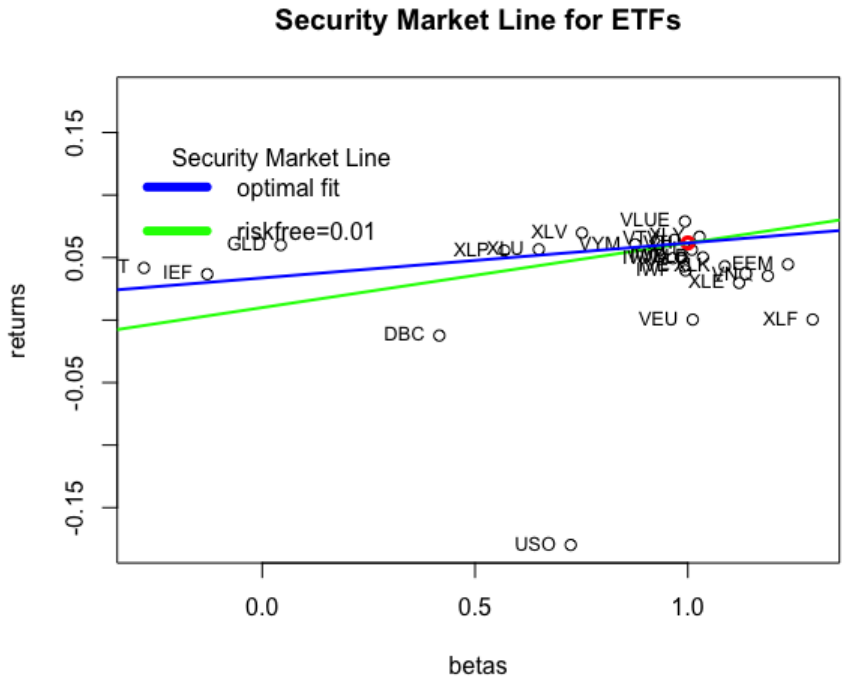
The *SML* depends on the choice of the risk-free rate r_f, with a steeper *SML* line for lower risk-free rates r_f.

All the different *SML* lines pass through the point $(\beta = 1, r = R_m)$ corresponding to the market, and they intersect the y-axis at the risk-free point $(\beta = 0, r = r_f)$.

A scatterplot of asset returns versus their β shows which assets earn a positive α, and which don't.

If an asset lies on the *SML*, then its returns are mostly *systematic*, and its α is equal to zero.

Assets above the *SML* have a positive α, and those below have a negative α.




```
> symbolv <- rownames(betam)
> betac <- betam[-match(c("VXX", "SVXY", "MTUM", "USMV", "QUAL"), s
> betac <- c(1, betac)
> names(betac)[1] <- "VTI"
> retsann <- sapply(retp[, names(betac)], PerformanceAnalytics::Ret
> # Plot scatterplot of returns vs betas
> minrets <- min(retsann)
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      ylim=c(minrets, -minrets), main="Security Market Line for E
> retvti <- retsann["VTI"]
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
> abline(a=raterf, b=(retvti-raterf), col="green", lwd=2)
```

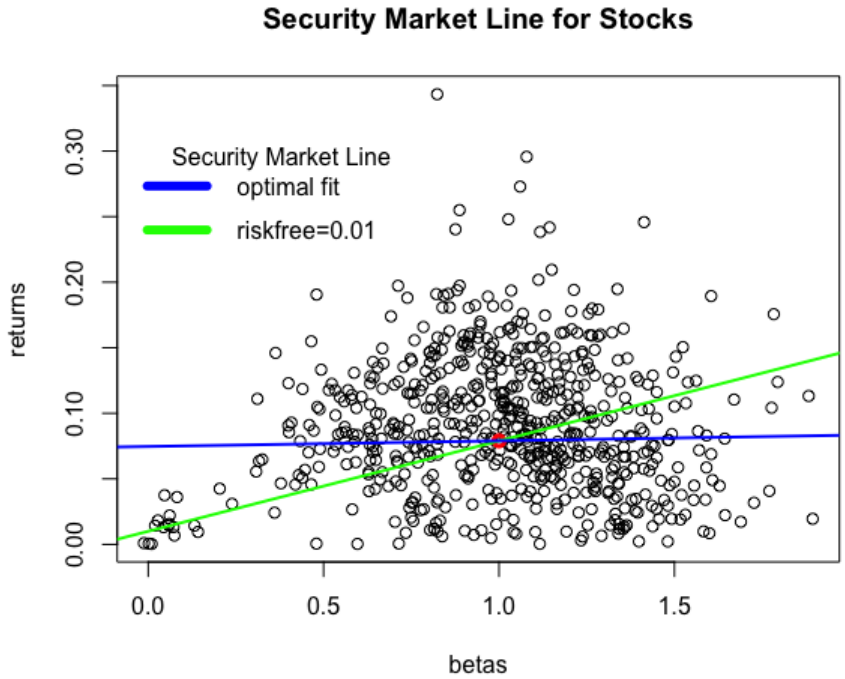
```
> # Add labels
> text(x=betac, y=retsann, labels=names(betac), pos=2, cex=0.8)
> # Find optimal risk-free rate by minimizing residuals
> rss <- function(raterf) {
+   sum((retsann - raterf - betac*(retvti-raterf))^2)
+ }  ##  end rss
> optimrss <- optimize(rss, c(-1, 1))
> raterf <- optimrss$minimum
> # Or simply
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

The Security Market Line for Stocks

The best fitting *Security Market Line* (SML) for stocks is almost flat, which shows that stocks with higher β don't earn higher returns.

This is called the *low beta anomaly*.

```
> # Load S&P500 constituent stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RDat
> retvti <- na.omit(rutils::etfenv$returns$VTI)
> retp <- retstock[index(retvti), ]
> nrows <- NROW(retp)
> # Calculate stock betas
> betac <- sapply(retp, function(x) {
+   retp <- na.omit(cbind(x, retvti))
+   drop(cov(retp[, 1], retp[, 2])/var(retp[, 2]))
+ })  ##  end sapply
> mean(betac)
> # Calculate annual stock returns
> retsann <- retp
> retsann[1, ] <- 0
> retsann <- zoo::na.locf(retsann, na.rm=FALSE)
> retsann <- 252*sapply(retsann, sum)/nrows
> # Remove stocks with zero returns
> sum(retsann == 0)
> betac <- betac[retsann > 0]
> retsann <- retsann[retsann > 0]
> retvti <- 252*mean(retvti)
> # Plot scatterplot of returns vs betas
> plot(retsann ~ betac, xlab="betas", ylab="returns",
+      main="Security Market Line for Stocks")
> points(x=1, y=retvti, col="red", lwd=3, pch=21)
> # Plot Security Market Line
> raterf <- 0.01
> abline(a=raterf, b=(retvti-raterf), col="green", lwd=2)
```




```
> # Find optimal risk-free rate by minimizing residuals
> retsadj <- (retsann - retvti*betac)
> betadj <- (1-betac)
> raterf <- sum(retsadj*betadj)/sum(betadj^2)
> abline(a=raterf, b=(retvti-raterf), col="blue", lwd=2)
> legend(x="topleft", bty="n", title="Security Market Line",
+  legend=c("optimal fit", "raterf=0.01"),
+  y.intersp=0.5, cex=1.0, lwd=6, lty=1, col=c("blue", "green"))
```

Beta-adjusted Performance Measurement

The *Treynor* ratio measures the excess returns per unit of the *systematic* risk *beta* β, and is equal to the excess returns (over a risk-free rate) divided by the β:

$$T_r = \frac{E[R - r_f]}{\beta}$$

The *Treynor* ratio is similar to the *Sharpe* ratio, with the difference that its denominator represents only *systematic* risk, not total risk.

The *Information* ratio is equal to the excess returns (over a benchmark) divided by the *tracking error* (standard deviation of excess returns):

$$I_r = \frac{E[R - R_b]}{\sqrt{\sum_{i=1}^{n}(R_i - R_{i,b})^2}}$$

The *Information* ratio measures the amount of outperformance versus the benchmark, and the consistency of outperformance.

```
> library(PerformanceAnalytics)
> # Calculate XLP Treynor ratio
> TreynorRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] 0.0982
> # Calculate XLP Information ratio
> InformationRatio(Ra=retp$XLP, Rb=retp$VTI)
[1] -0.0813
```

CAPM Summary Statistics

`PerformanceAnalytics::table.CAPM()` calculates the *beta* β and *alpha* α values, the *Treynor* ratio, and other performance statistics.

```
> PerformanceAnalytics::table.CAPM(Ra=retp[, c("XLP", "XLF")],
+                           Rb=retp$VTI, scale=252)
                   XLP to VTI XLF to VTI
Alpha                  0.0001    -0.0002
Beta                   0.5493     1.2500
Alpha Robust           0.0001    -0.0003
Beta Robust            0.5467     1.0780
Beta+                  0.5640     1.3181
Beta-                  0.5653     1.3259
Beta+ Robust           0.5441     1.0962
Beta- Robust           0.5529     1.1177
R-squared              0.5323     0.7248
R-squared Robust       0.4311     0.6227
Annualized Alpha       0.0224    -0.0566
Correlation            0.7296     0.8513
Correlation p-value    0.0000     0.0000
Tracking Error         0.1316     0.1557
Active Premium        -0.0107    -0.0599
Information Ratio     -0.0813    -0.3849
Treynor Ratio          0.0982     0.0136
```

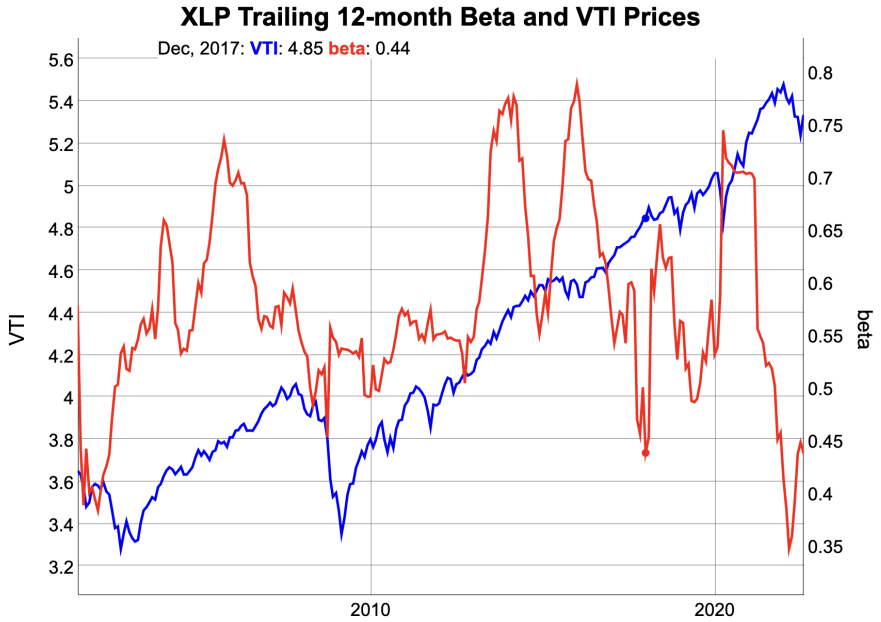
```
> capmstats <- table.CAPM(Ra=retp[, symbolv],
+         Rb=retp$VTI, scale=252)
> colv <- strsplit(colnames(capmstats), split=" ")
> colv <- do.call(cbind, colv)[1, ]
> colnames(capmstats) <- colv
> capmstats <- t(capmstats)
> capmstats <- capmstats[, -1]
> colv <- colnames(capmstats)
> whichv <- match(c("Annualized Alpha", "Information Ratio", "Treynor Rat
> colv[whichv] <- c("Alpha", "Information", "Treynor")
> colnames(capmstats) <- colv
> capmstats <- capmstats[order(capmstats[, "Alpha"], decreasing=TRUE), ]
> # Copy capmstats into etfenv and save to .RData file
> etfenv <- rutils::etfenv
```

```
> rutils::etfenv$capmstats[, c("Beta", "Alpha", "Information"
        Beta   Alpha Information Treynor
GLD   0.0497  0.1040      0.0240  1.8623
TLT  -0.2363  0.0657     -0.2370 -0.1202
IEF  -0.1106  0.0477     -0.2710 -0.2983
XLU   0.6123  0.0276     -0.0268  0.0977
XLV   0.7343  0.0204      0.0082  0.0891
XLP   0.5451  0.0203     -0.0737  0.0989
XLY   1.0119  0.0121      0.0526  0.0697
USMV  0.7261  0.0116     -0.3225  0.1502
XLI   0.9658  0.0096      0.0321  0.0700
QQQ   1.1889  0.0049      0.0148  0.0543
VTI   0.9947  0.0031      0.0978  0.0737
IVW   0.9946  0.0031      0.0220  0.0650
IWB   0.9785  0.0022      0.0189  0.0644
MTUM  1.0295  0.0019      0.0189  0.1252
SPY   1.0000  0.0000         NaN  0.0873
XLB   0.9448 -0.0005     -0.0996  0.0525
XLK   1.1588 -0.0015     -0.0258  0.0527
IWF   1.0315 -0.0015     -0.0521  0.0569
IWD   0.9429 -0.0022     -0.1104  0.0595
QUAL  0.9899 -0.0023     -0.1308  0.1182
VYM   0.8640 -0.0025     -0.1907  0.0806
XLE   0.9830 -0.0052     -0.1440  0.0338
IVE   0.9560 -0.0055     -0.1560  0.0558
VTV   0.9455 -0.0073     -0.2120  0.0759
VLUE  0.9777 -0.0311     -0.5118  0.0880
DBC   0.4005 -0.0350     -0.4748 -0.0327
EEM   1.1813 -0.0380     -0.2563  0.0452
XLF   1.2175 -0.0408     -0.2904  0.0140
VNQ   1.1359 -0.0446     -0.3049  0.0268
VEU   0.9870 -0.0530     -0.6364  0.0226
AIEQ  1.1573 -0.0680     -0.6789  0.1157
USO   0.6925 -0.1628     -0.7137 -0.2334
SVXY  2.1496 -0.1841         NaN     NaN
VXX  -2.8693 -0.2710     -0.9246  0.2127
```

Trailing Stock Beta Over Time

The trailing beta of *XLP* versus *VTI* changes over time, with lower beta in periods of stock selloffs.

The function `roll_reg()` from package *HighFreq* performs trailing regressions in `C++` (*RcppArmadillo*), so it's therefore much faster than equivalent `R` code.



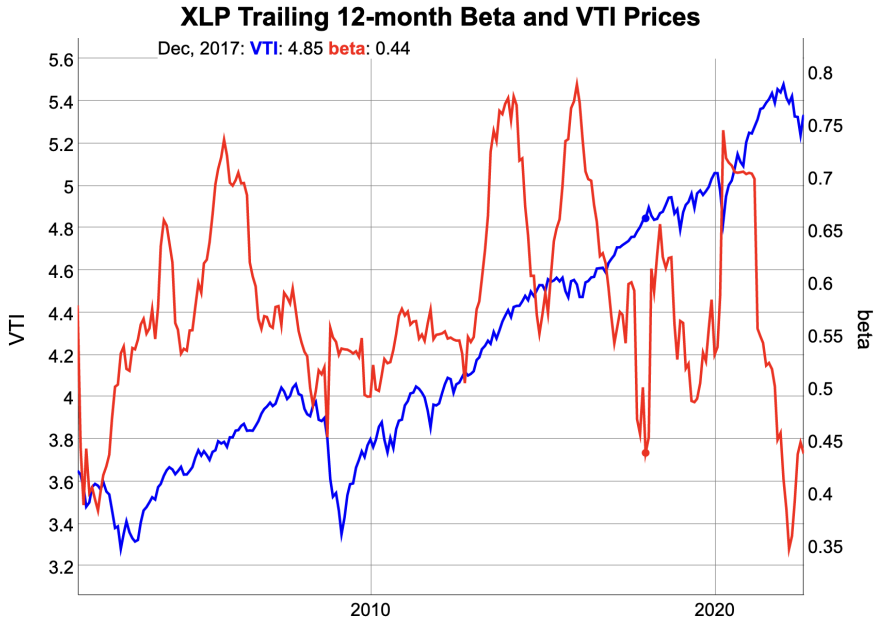

```
> # Calculate XLP and VTI returns
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> # Calculate monthly end points
> endd <- xts::endpoints(retp, on="months")[-1]
> # Calculate start points from look-back interval
> lookb <- 12  ##  Look back 12 months
> startp <- c(rep(1, lookb), endd[1:(NROW(endd)-lookb)])
> head(cbind(endd, startp), lookb+2)
> # Calculate trailing beta regressions every month in R
> formulav <- XLP ~ VTI  ##  Specify regression formula
> betar <- sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     ##  coef(lm(formulav, data=datav))[2]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   })  ##  end sapply
> # Calculate trailing betas using RcppArmadillo
> controll <- HighFreq::param_reg()
> reg_stats <- HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI,
+   startp=(startp-1), endp=(endd-1), controll=controll)
> betac <- reg_stats[, 2]
> all.equal(betac, betar)
> # Compare the speed of RcppArmadillo with R code
> library(microbenchmark)
> summary(microbenchmark(
+   Rcpp=HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI, startp=(startp-1), endp=(endd-1), controll=controll),
+   Rcode=sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   }),
```

```
> # dygraph plot of trailing XLP beta and VTI prices
> datev <- zoo::index(retp[endd, ])
> pricev <- rutils::etfenv$prices$VTI[datev]
> datav <- cbind(pricev, betac)
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing 12-month Beta and VTI
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue") %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```

Trailing Stock Beta Over Time

The trailing beta of *XLP* versus *VTI* changes over time, with lower beta in periods of stock selloffs.

The function `roll_reg()` from package *HighFreq* performs trailing regressions in C++ (*RcppArmadillo*), so it's therefore much faster than equivalent R code.




```
> # Calculate XLP and VTI returns
> retp <- na.omit(rutils::etfenv$returns[, c("XLP", "VTI")])
> # Calculate monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")[-1]
> # Calculate start points from look-back interval
> lookb <- 12  ##  Look back 12 months
> startp <- c(rep(1, lookb), endd[1:(NROW(endd)-lookb)])
> head(cbind(endd, startp), lookb+2)
> # Calculate trailing beta regressions every month in R
> formulav <- XLP ~ VTI  ##  Specify regression formula
> betar <- sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     ##  coef(lm(formulav, data=datav))[2]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   })  ##  end sapply
> # Calculate trailing betas using RcppArmadillo
> controll <- HighFreq::param_reg()
> reg_stats <- HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI,
+   startp=(startp-1), endp=(endd-1), controll=controll)
> betac <- reg_stats[, 2]
> all.equal(betac, betar)
> # Compare the speed of RcppArmadillo with R code
> library(microbenchmark)
> summary(microbenchmark(
+   Rcpp=HighFreq::roll_reg(respv=retp$XLP, predm=retp$VTI, startp=(startp-1), endp=(endd-1), controll=controll),
+   Rcode=sapply(1:NROW(endd), FUN=function(tday) {
+     datav <- retp[startp[tday]:endd[tday], ]
+     drop(cov(datav$XLP, datav$VTI)/var(datav$VTI))
+   }),
```

```
> # dygraph plot of trailing XLP beta and VTI prices
> datev <- zoo::index(retp[endd, ])
> pricev <- log(rutils::etfenv$prices$VTI[datev])
> datav <- cbind(pricev, betac)
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing 12-month Beta and VTI
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```

Recursive Trailing Stock Beta

The trailing beta β of a stock with returns r_t with respect to a stock index with returns R_t can be updated using these recursive formulas with the decay factor λ:

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1-\lambda) r_t$$

$$\bar{R}_t = \lambda \bar{R}_{t-1} + (1-\lambda) R_t$$

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)(R_t - \bar{R}_t)^2$$

$$\text{cov}_t = \lambda \, \text{cov}_{t-1} + (1-\lambda)(r_t - \bar{r}_t)(R_t - \bar{R}_t)$$

$$\beta_t = \frac{\text{cov}_t}{\sigma_t^2}$$

The parameter λ determines the rate of decay of the weight of past returns. If λ is close to 1 then the decay is weak and past returns have a greater weight, and the trailing mean values have a stronger dependence on past returns. This is equivalent to a long look-back interval. And vice versa if λ is close to 0.

The function `HighFreq::run_covar()` calculates the trailing variances, covariances, and means of two *time series*.

```
> # Calculate the trailing betas
> lambdaf <- 0.99
> covarv <- HighFreq::run_covar(retp, lambdaf)
> betac <- covarv[, 1]/covarv[, 3]
```


```
> # dygraph plot of trailing XLP beta and VTI prices
> datav <- cbind(pricev, betac[endd])[-(1:11)] ##  Remove warmup per
> colnames(datav)[2] <- "beta"
> colv <- colnames(datav)
> dygraphs::dygraph(datav, main="XLP Trailing 12-month Beta and VTI
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="blue", strokeWidth=2) %>%
+   dySeries(name=colv[2], axis="y2", col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```

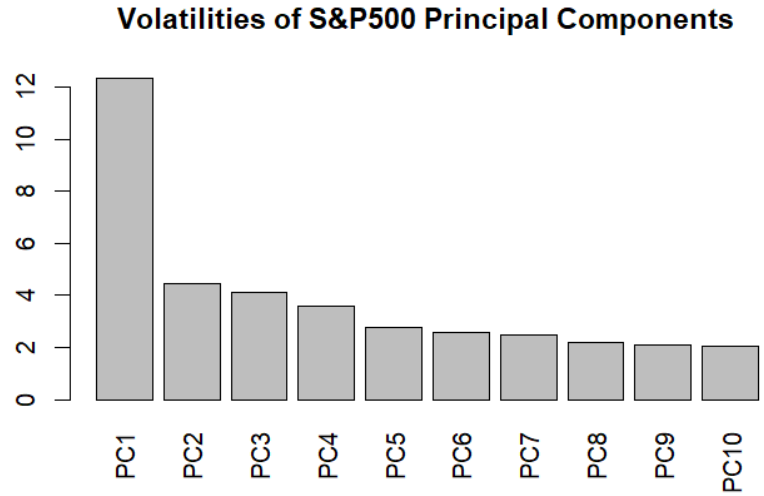
Principal Components of *S&P500* Stock Constituents

The *PCA* standard deviations are the volatilities of the *principal component* time series.

The original time series of returns can be calculated approximately from the first few *principal components* with the largest standard deviations.

The *Kaiser-Guttman* rule uses only *principal components* with *variance* greater than 1.

Another rule of thumb is to use the *principal components* with the largest standard deviations which sum up to 80% of the total variance of returns.




```
> # Load S&P500 constituent stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_prices.RData'
> # Calculate stock prices and percentage returns
> pricets <- zoo::na.locf(pricets, na.rm=FALSE)
> pricets <- zoo::na.locf(pricets, fromLast=TRUE)
> retp <- rutils::diffit(log(pricev))
> # Standardize (center and scale) the returns
> retp <- lapply(retp, function(x) {(x - mean(x))/sd(x)})
> retp <- rutils::do_call(cbind, retp)
> # Perform principal component analysis PCA
> pcad <- prcomp(retp, scale=TRUE)
> # Find number of components with variance greater than 2
> ncomp <- which(pcad$sdev^2 < 2)[1]
```

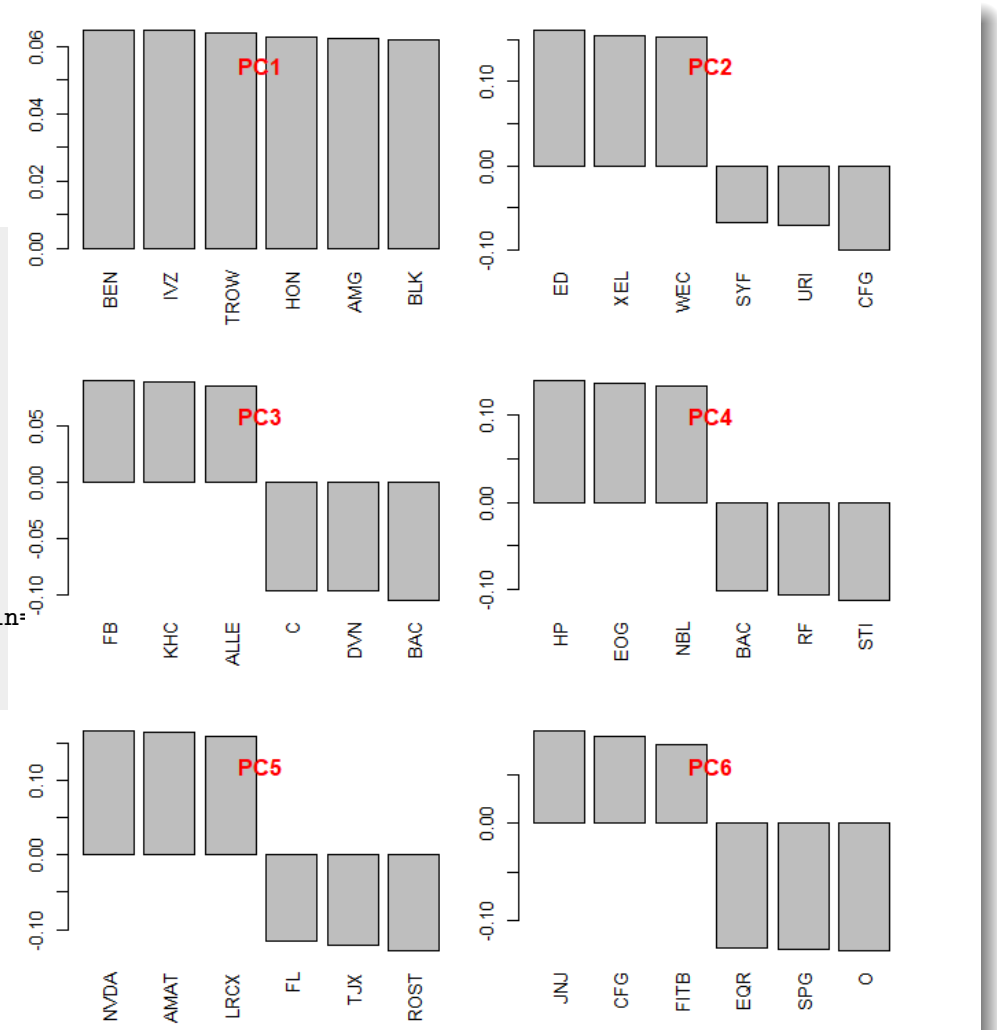
```
> # Plot standard deviations of principal components
> barplot(pcad$sdev[1:ncomp],
+   names.arg=colnames(pcad$rotation[, 1:ncomp]),
+   las=3, xlab="", ylab="",
+   main="Volatilities of S&P500 Principal Components")
```

S&P500 Principal Component Loadings

Principal component loadings are the weights of *principal component* portfolios.

The *principal component* portfolios have mutually orthogonal returns represent the different orthogonal modes of the return variance.

```
> # Calculate principal component loadings (weights)
> # Plot barplots with PCA weights in multiple panels
> ncomps <- 6
> par(mfrow=c(ncomps/2, 2))
> par(mar=c(4, 2, 2, 1), oma=c(0, 0, 0, 0))
> # First principal component weights
> weightv <- sort(pcad$rotation[, 1], decreasing=TRUE)
> barplot(weightv[1:6], las=3, xlab="", ylab="", main="")
> title(paste0("PC", 1), line=-2.0, col.main="red")
> for (ordern in 2:ncomps) {
+   weightv <- sort(pcad$rotation[, ordern], decreasing=TRUE)
+   barplot(weightv[c(1:3, 498:500)], las=3, xlab="", ylab="", main=
+   title(paste0("PC", ordern), line=-2.0, col.main="red")
+ }  ##  end for
```

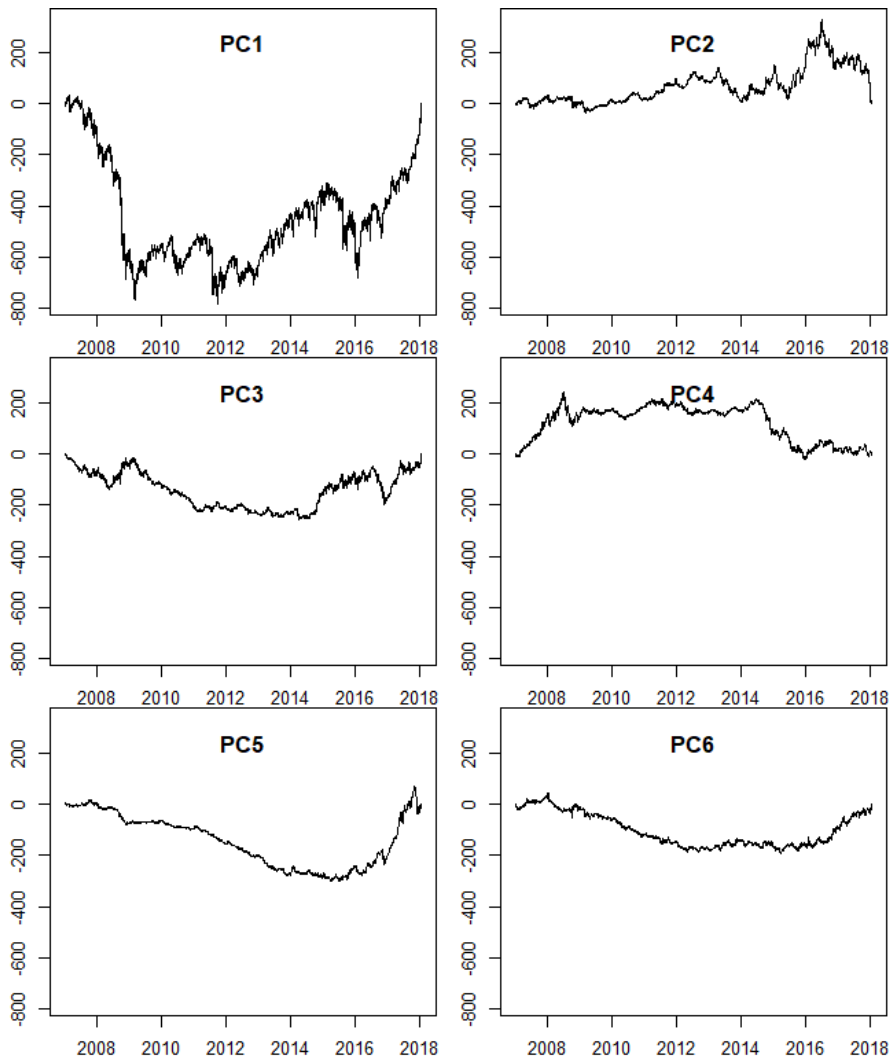


S&P500 Principal Component Time Series

The time series of the *principal components* can be calculated by multiplying the loadings (weights) times the original data.

Higher order *principal components* are gradually less volatile.

```
> # Calculate principal component time series
> retpca <- xts(retp %*% pcad$rotation[, 1:ncomps], order.by=datev)
> round(cov(retpca), 3)
> retpcac <- cumsum(retpca)
> # Plot principal component time series in multiple panels
> par(mfrow=c(ncomps/2, 2))
> par(mar=c(2, 2, 0, 1), oma=c(0, 0, 0, 0))
> rangev <- range(retpcac)
> for (ordern in 1:ncomps) {
+   plot.zoo(retpcac[, ordern], ylim=rangev, xlab="", ylab="")
+   title(paste0("PC", ordern), line=-2.0)
+ }  ##  end for
```



S&P500 Factor Model From *Principal Components*

By inverting the *PCA* analysis, the *S&P500* constituent returns can be calculated from the first `k` *principal components* under a *factor model*:

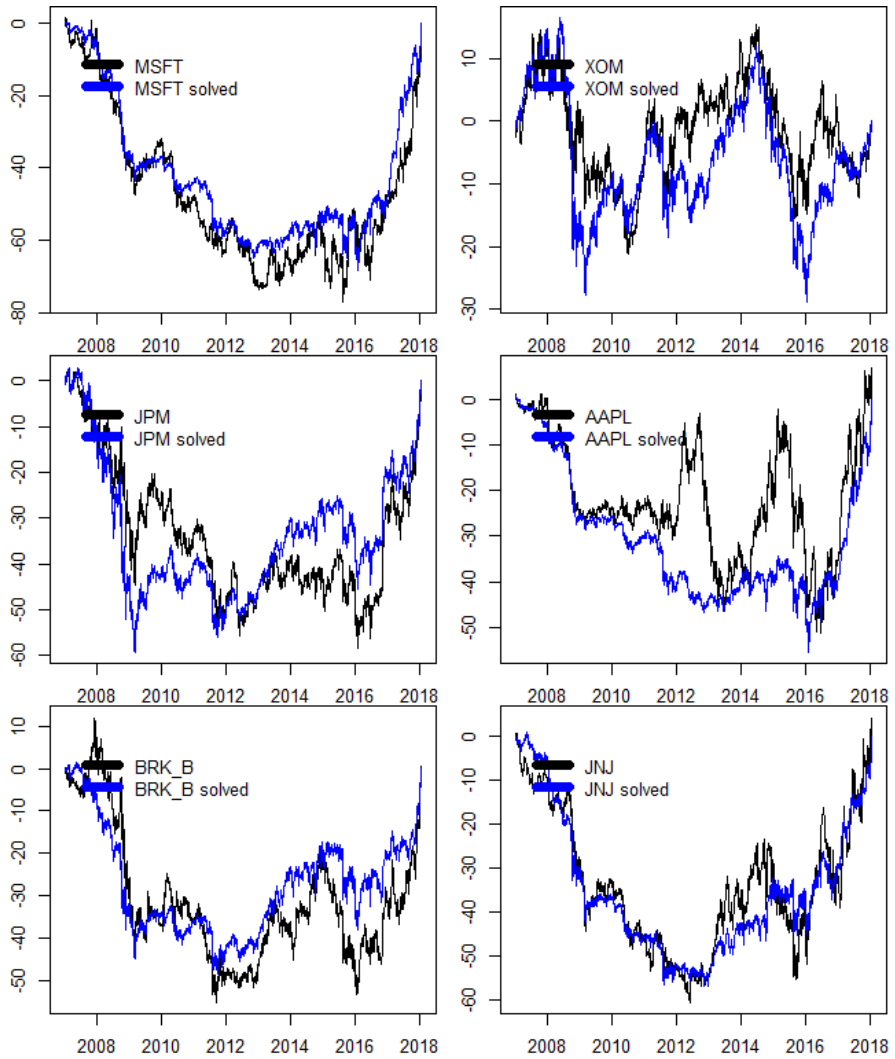
$$\mathbf{r}_i = \alpha_i + \sum_{j=1}^{k} \beta_{ji} \mathbf{F}_j + \varepsilon_i$$

The *principal components* are interpreted as *market factors*: $\mathbf{F}_j = \mathbf{pc}_j$.

The *market betas* are the inverse of the *principal component loadings*: $\beta_{ji} = w_{ij}$.

The ε_i are the *idiosyncratic* returns, which should be mutually independent and uncorrelated to the *market factor* returns.

```
> # Invert principal component time series
> pcinv <- solve(pcad$rotation)
> all.equal(pcinv, t(pcad$rotation))
> solved <- retpca %*% pcinv[1:ncomps, ]
> solved <- xts::xts(solved, datev)
> solved <- cumsum(solved)
> retc <- cumsum(retp)
> # Plot the solved returns
> symbolv <- c("MSFT", "XOM", "JPM", "AAPL", "BRK_B", "JNJ")
> for (symbol in symbolv) {
+   plot.zoo(cbind(retc[, symbol], solved[, symbol]),
+     plot.type="single", col=c("black", "blue"), xlab="", ylab="")
+   legend(x="topleft", bty="n",
+    legend=paste0(symbol, c("", " solved")),
+    title=NULL, inset=0.05, cex=1.0, lwd=6,
+    lty=1, col=c("black", "blue"))
+ }  ##  end for
```



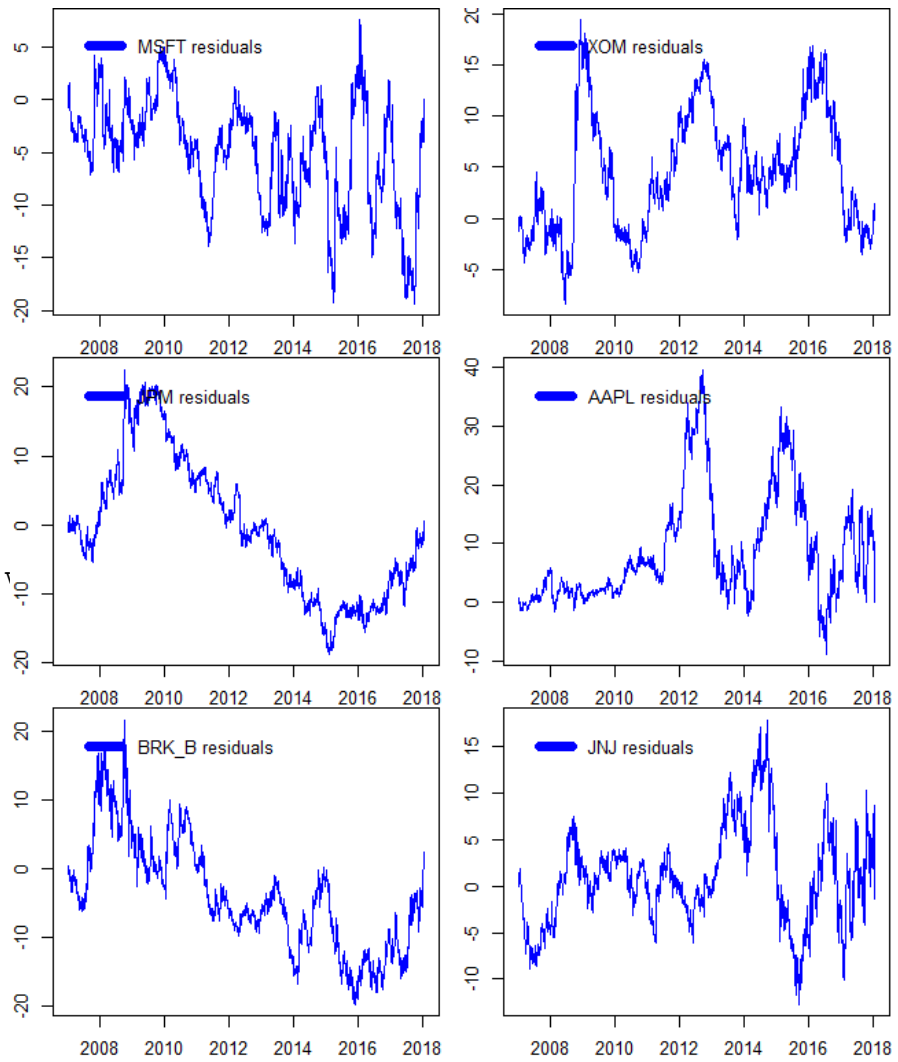
S&P500 Factor Model Residuals

The original time series of returns can be calculated exactly from the time series of all the *principal components*, by inverting the loadings matrix.

The original time series of returns can be calculated approximately from just the first few *principal components*, which demonstrates that *PCA* is a form of *dimension reduction*.

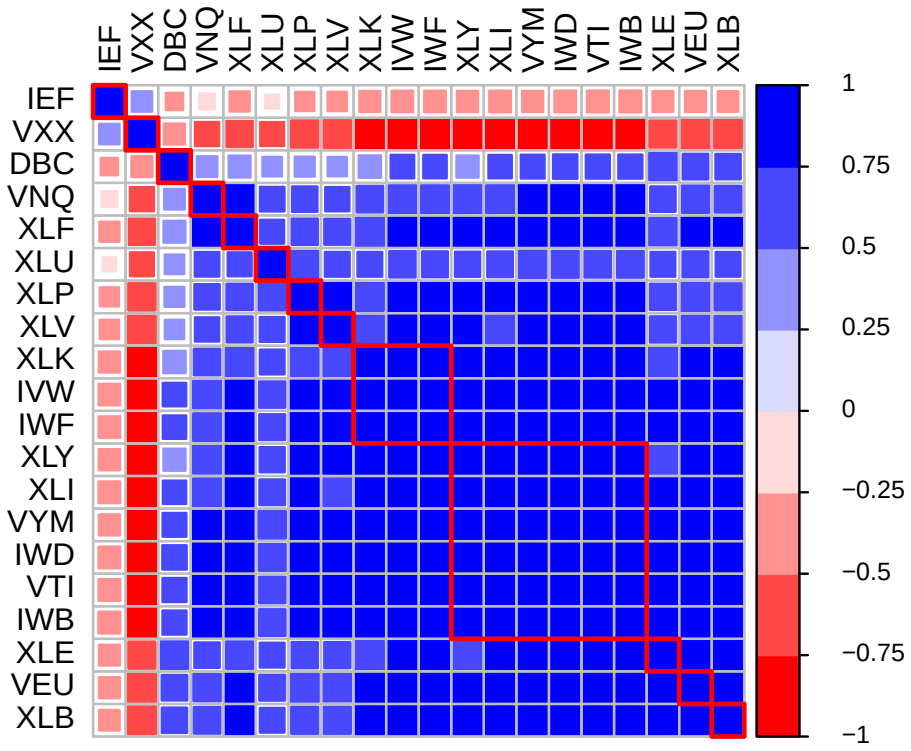
The function `solve()` solves systems of linear equations, and also inverts square matrices.

```
> # Perform ADF unit root tests on original series and residuals
> sapply(symbolv, function(symbol) {
+   c(series=tseries::adf.test(retc[, symbol])$p.value,
+     resid=tseries::adf.test(retc[, symbol] - solved[, symbol])$p.v
+ })  ##  end sapply
> # Plot the residuals
> for (symbol in symbolv) {
+   plot.zoo(retc[, symbol] - solved[, symbol],
+     plot.type="single", col="blue", xlab="", ylab="")
+   legend(x="topleft", bty="n", legend=paste(symbol, "residuals"),
+    title=NULL, inset=0.05, cex=1.0, lwd=6, lty=1, col="blue")
+ }  ##  end for
> # Perform ADF unit root test on principal component time series
> retpca <- xts(retp %*% pcad$rotation, order.by=datev)
> retpcac <- cumsum(retpca)
> adf_pvalues <- sapply(1:NCOL(retpcac), function(ordern)
+   tseries::adf.test(retpcac[, ordern])$p.value)
> # AdF unit root test on stationary time series
> tseries::adf.test(rnorm(1e5))
```



Correlation and Factor Analysis

```
> ### Perform pair-wise correlation analysis
> # Calculate correlation matrix
> cormat <- cor(retp)
> colnames(cormat) <- colnames(retp)
> rownames(cormat) <- colnames(retp)
> # Reorder correlation matrix based on clusters
> # Calculate permutation vector
> library(corrplot)
> ordern <- corrMatOrder(cormat, order="hclust",
+         hclust.method="complete")
> # Apply permutation vector
> cormat <- cormat[ordern, ordern]
> # Plot the correlation matrix
> colorv <- colorRampPalette(c("red", "white", "blue"))
> corrplot(cormat, tl.col="black", tl.cex=0.8,
+     method="square", col=colorv(8),
+     cl.offset=0.75, cl.cex=0.7,
+     cl.align.text="l", cl.ratio=0.25)
> # draw rectangles on the correlation matrix plot
> corrRect.hclust(cormat, k=NROW(cormat) %/% 2,
+           method="complete", col="red")
```



Hierarchical Clustering Analysis

The function `as.dist()` converts a matrix representing the *distance* (dissimilarity) between elements, into a list of class `"dist"`.

For example, `as.dist()` converts `(1-correlation)` to distance.

The function `hclust()` recursively combines elements into clusters based on their mutual *distance*.

First `hclust()` combines individual elements that are closest to each other.

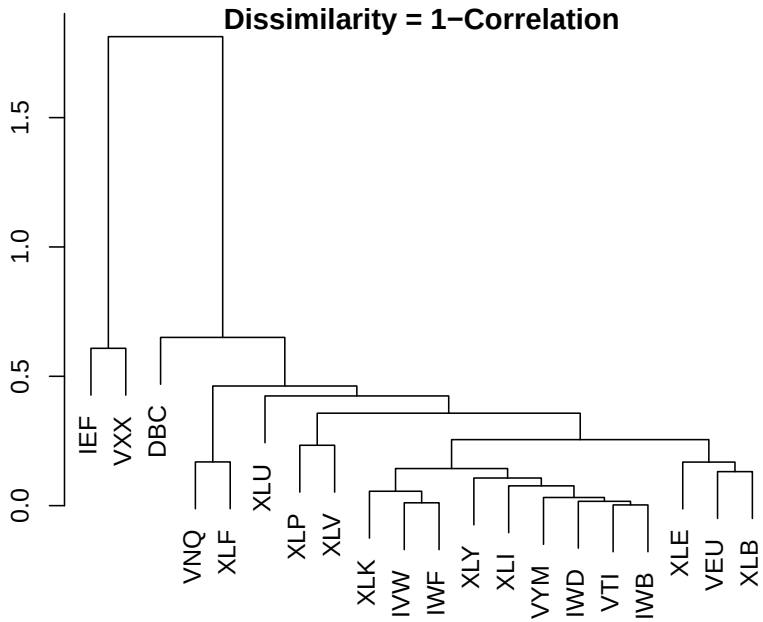
Then it combines elements to the closest clusters, then clusters with other clusters, until all elements are combined into one cluster.

This process of recursive clustering can be represented as a *dendrogram* (tree diagram).

Branches of a *dendrogram* represent clusters.

Neighboring branches contain elements that are close to each other (have small distance).

Neighboring branches combine into larger branches, that then combine with their closest branches, etc.




```
> # Convert correlation matrix into distance object
> distancev <- as.dist(1-cormat)
> # Perform hierarchical clustering analysis
> compclust <- hclust(distancev)
> plot(compclust, ann=FALSE, xlab="", ylab="")
> title("Dendrogram representing hierarchical clustering
+ \nwith dissimilarity = 1-correlation", line=-0.5)
```

Autoregressive Processes

An *autoregressive* process *AR(n)* of order n for a time series r_t is defined as:

$$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \ldots + \varphi_n r_{t-n} + \xi_t$$

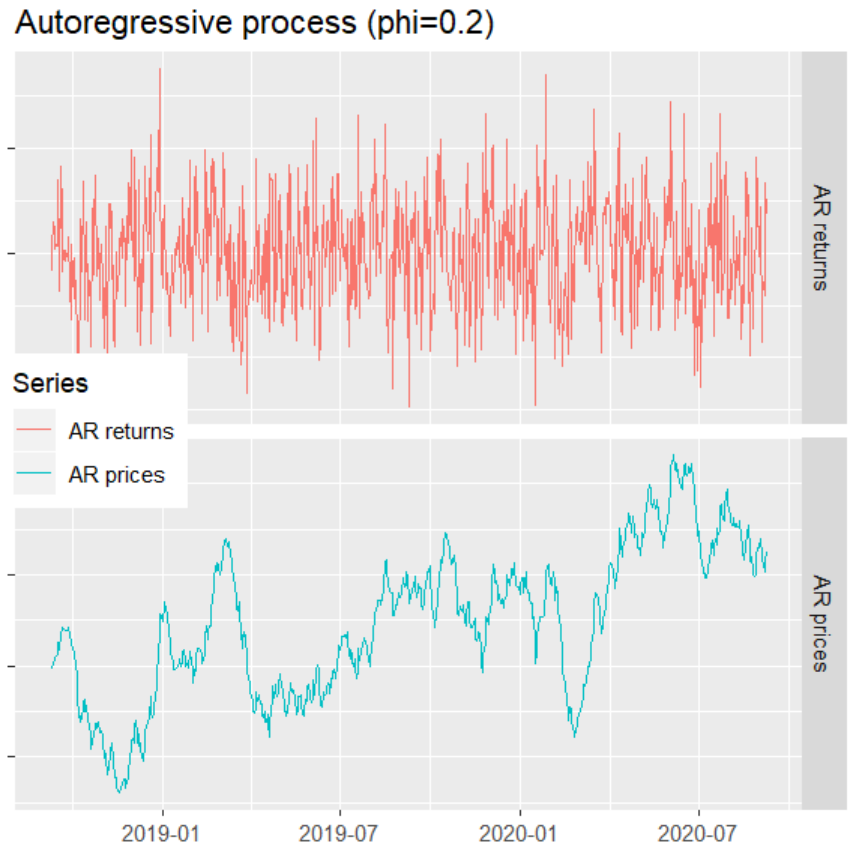
Where φ_i are the *AR(n)* coefficients, and ξ_t are standard normal *innovations*.

The *AR(n)* process is a special case of an *ARIMA* process, and is simply called an *AR(n)* process.

If the *AR(n)* process is *stationary* then the time series r_t is mean reverting to zero.

The function `arima.sim()` simulates *ARIMA* processes, with the `"model"` argument accepting a `list` of *AR(n)* coefficients φ_i.

```
> # Simulate AR processes
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ## I
> datev <- Sys.Date() + 0:728  ##  Two year daily series
> # AR time series of returns
> arimav <- xts(x=arima.sim(n=NROW(datev), model=list(ar=0.2)),
+           order.by=datev)
> arimav <- cbind(arimav, cumsum(arimav))
> colnames(arimav) <- c("AR returns", "AR prices")
```



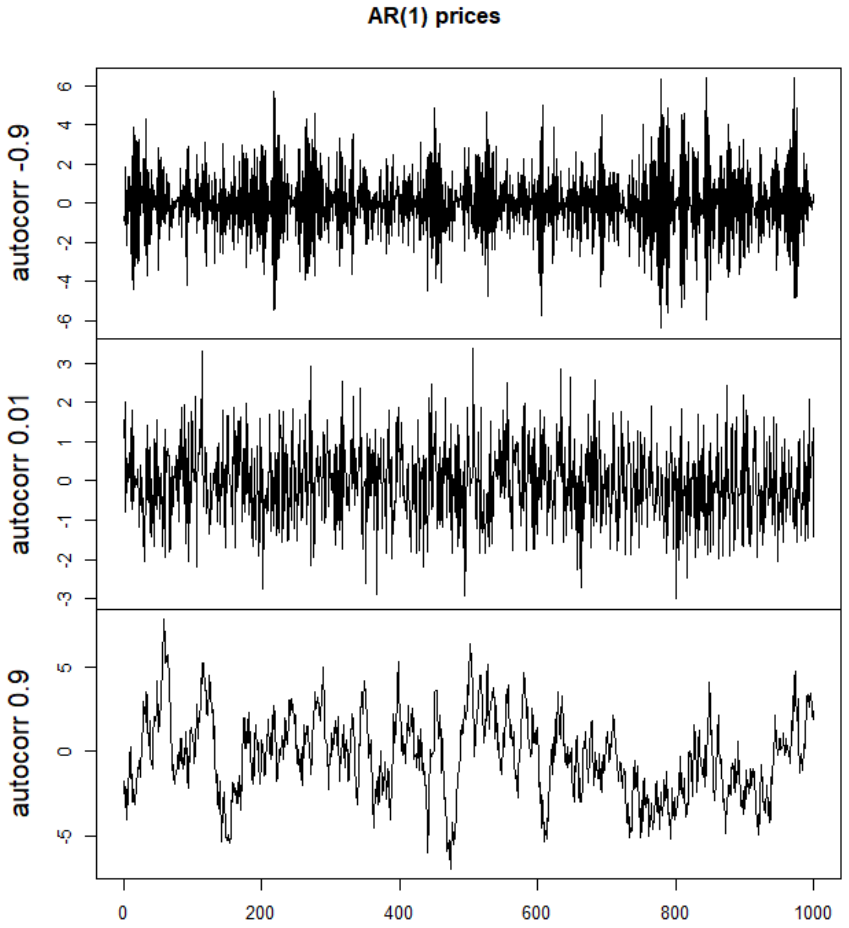

```
> library(ggplot2)  ##  Load ggplot2
> library(gridExtra)  ##  Load gridExtra
> autoplot(object=arimav, ##  ggplot AR process
+  facets="Series ~ .",
+  main="Autoregressive process (phi=0.2)") +
+   facet_grid("Series ~ .", scales="free_y") +
+   xlab("") + ylab("") +
+ theme(legend.position=c(0.1, 0.5),
+   plot.background=element_blank(),
+   axis.text.y=element_blank())
```

Examples of Autoregressive Processes

The speed of mean reversion of an *AR(1)* process depends on the *AR(n)* coefficient φ_1, with a negative coefficient producing faster mean reversion, and a positive coefficient producing stronger diversion.

A positive coefficient φ_1 produces a diversion away from the mean, so that the time series r_t wanders away from the mean for longer periods of time.

```
> coeff <- c(-0.9, 0.01, 0.9)  ##  AR coefficients
> # Create three AR time series
> arimav <- sapply(coeff, function(phi) {
+   set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  #
+   arima.sim(n=NROW(datev), model=list(ar=phi))
+ })  ##  end sapply
> colnames(arimav) <- paste("autocorr", coeff)
> plot.zoo(arimav, main="AR(1) prices", xlab=NA)
> # Or plot using ggplot
> arimav <- xts(x=arimav, order.by=datev)
> library(ggplot)
> autoplot(arimav, main="AR(1) prices",
+    facets=Series ~ .) +
+     facet_grid(Series ~ ., scales="free_y") +
+ xlab("") +
+ theme(
+   legend.position=c(0.1, 0.5),
+   plot.title=element_text(vjust=-2.0),
+   plot.margin=unit(c(-0.5, 0.0, -0.5, 0.0), "cm"),
+   plot.background=element_blank(),
+   axis.text.y=element_blank())
```



Simulating Autoregressive Processes

An *autoregressive* process *AR(n)*:

$$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \ldots + \varphi_n r_{t-n} + \xi_t$$

Can be simulated by using an explicit recursive loop in R.

AR(n) processes can also be simulated by using the function `filter()` directly, with the argument `method="recursive"`.

The function `filter()` applies a linear filter to a vector, and returns a time series of class `"ts"`.

The function `HighFreq::sim_ar()` simulates an *AR(n)* processes using C++ code.

```
> # Define AR(3) coefficients and innovations
> coeff <- c(0.1, 0.39, 0.5)
> nrows <- 1e2
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection"); innov
> # Simulate AR process using recursive loop in R
> arimav <- numeric(nrows)
> arimav[1] <- innov[1]
> arimav[2] <- coeff[1]*arimav[1] + innov[2]
> arimav[3] <- coeff[1]*arimav[2] + coeff[2]*arimav[1] + innov[3]
> for (it in 4:NROW(arimav)) {
+   arimav[it] <- arimav[(it-1):(it-3)] %*% coeff + innov[it]
+ }  ##  end for
> # Simulate AR process using filter()
> arimaf <- filter(x=innov, filter=coeff, method="recursive")
> class(arimaf)
> all.equal(arimav, as.numeric(arimaf))
> # Fast simulation of AR process using C_rfilter()
> arimacpp <- .Call(stats:::C_rfilter, innov, coeff,
+      double(NROW(coeff) + NROW(innov)))[-(1:3)]
> all.equal(arimav, arimacpp)
> # Fastest simulation of AR process using HighFreq::sim_ar()
> arimav <- HighFreq::sim_ar(coeff=matrix(coeff), innov=matrix(innov
> arimav <- drop(arimav)
> all.equal(arimav, arimacpp)
> # Benchmark the speed of the three methods of simulating AR proces
> library(microbenchmark)
> summary(microbenchmark(
+   Rloop={for (it in 4:NROW(arimav)) {
+     arimav[it] <- arimav[(it-1):(it-3)] %*% coeff + innov[it]
+   }},
+   filter=filter(x=innov, filter=coeff, method="recursive"),
+   cpp=HighFreq::sim_ar(coeff=matrix(coeff), innov=matrix(innov))
+   ), times=10)[, c(1, 4, 5)]
```

Simulating Autoregressive Processes Using arima.sim()

The function `arima.sim()` simulates *ARIMA* processes by calling the function `filter()`.

ARIMA processes can also be simulated by using the function `filter()` directly, with the argument `method="recursive"`.

Simulating stationary *autoregressive* processes requires a *warmup period*, to allow the process to reach its stationary state.

The required length of the *warmup period* depends on the smallest root of the characteristic equation, with a longer *warmup period* needed for smaller roots, that are closer to 1.

The *rule of thumb* (heuristic rule, guideline) is for the *warmup period* to be equal to 6 divided by the logarithm of the smallest characteristic root plus the number of *AR(n)* coefficients: $\frac{6}{\log(minroot)} + \text{numcoeff}$

```
> # Calculate modulus of roots of characteristic equation
> rootv <- Mod(polyroot(c(1, -coeff)))
> # Calculate warmup period
> warmup <- NROW(coeff) + ceiling(6/log(min(rootv)))
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nrows <- 1e4
> innov <- rnorm(nrows + warmup)
> # Simulate AR process using arima.sim()
> arimav <- arima.sim(n=nrows,
+   model=list(ar=coeff),
+   start.innov=innov[1:warmup],
+   innov=innov[(warmup+1):NROW(innov)])
> # Simulate AR process using filter()
> arimaf <- filter(x=innov, filter=coeff, method="recursive")
> all.equal(arimaf[-(1:warmup)], as.numeric(arimav))
> # Benchmark the speed of the three methods of simulating AR proces
> library(microbenchmark)
> summary(microbenchmark(
+   filter=filter(x=innov, filter=coeff, method="recursive"),
+   arima_sim=arima.sim(n=nrows,
+                       model=list(ar=coeff),
+                       start.innov=innov[1:warmup],
+                       innov=innov[(warmup+1):NROW(innov)]),
+   arima_loop={for (it in 4:NROW(arimav)) {
+   arimav[it] <- arimav[(it-1):(it-3)] %*% coeff + innov[it]}}
+   ), times=10)[, c(1, 4, 5)]
```

Autocorrelations of Autoregressive Processes

The autocorrelation ρ_i of an *AR(1)* process (defined as $r_t = \varphi r_{t-1} + \xi_t$), satisfies the recursive equation: $\rho_i = \varphi \rho_{i-1}$, with $\rho_1 = \varphi$.

Therefore *AR(1)* processes have exponentially decaying autocorrelations: $\rho_i = \varphi^i$.

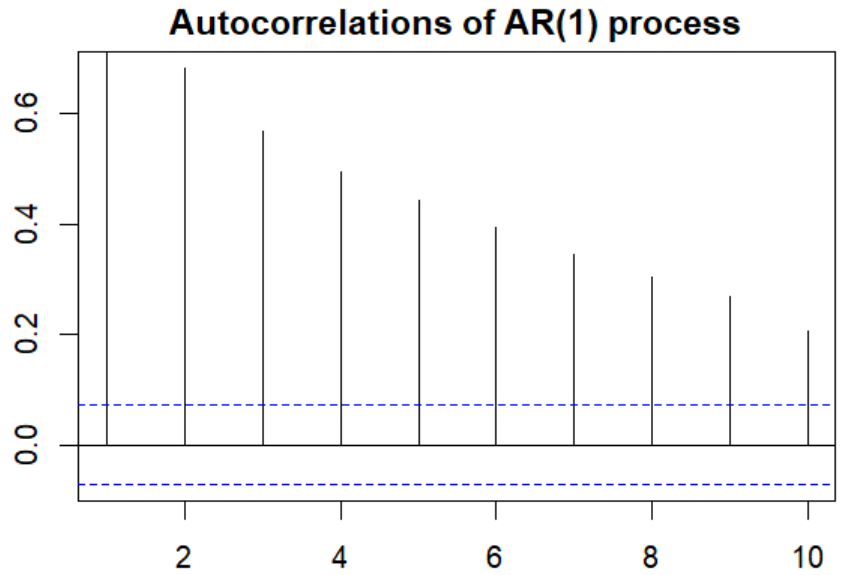
The *AR(1)* process can be simulated recursively:

$$r_1 = \xi_1$$
$$r_2 = \varphi r_1 + \xi_2 = \xi_2 + \varphi \xi_1$$
$$r_3 = \xi_3 + \varphi \xi_2 + \varphi^2 \xi_1$$
$$r_4 = \xi_4 + \varphi \xi_3 + \varphi^2 \xi_2 + \varphi^3 \xi_1$$

Therefore the *AR(1)* process can be expressed as a *moving average* (*MA*) of the *innovations* ξ_t: $r_t = \sum_{i=1}^{n} \varphi^{i-1} \xi_t$.

If $\varphi < 1.0$ then the influence of the innovation ξ_t decays exponentially.

If $\varphi = 1.0$ then the influence of the random innovations ξ_t persists indefinitely, so that the variance of r_t is proportional to time.



An *AR(1)* process has an exponentially decaying *ACF*.

```
> x11(width=6, height=4)
> par(mar=c(3, 3, 2, 1), oma=c(0, 0, 0, 0))
> # Simulate AR(1) process
> arimav <- arima.sim(n=1e3, model=list(ar=0.8))
> # ACF of AR(1) process
> acfl <- rutils::plot_acf(arimav, lag=10, xlab="", ylab="",
+   main="Autocorrelations of AR(1) process")
> acfl$acf[1:5]
```

Partial Autocorrelations

An autocorrelation of lag 1 induces higher order autocorrelations of lag `2, 3, ...`, which may obscure the direct higher order autocorrelations.

If two random variables are both correlated to a third variable, then they are indirectly correlated with each other.

The indirect correlation can be removed by defining new variables with no correlation to the third variable.

The *partial correlation* is the correlation after the correlations to the common variables are removed.

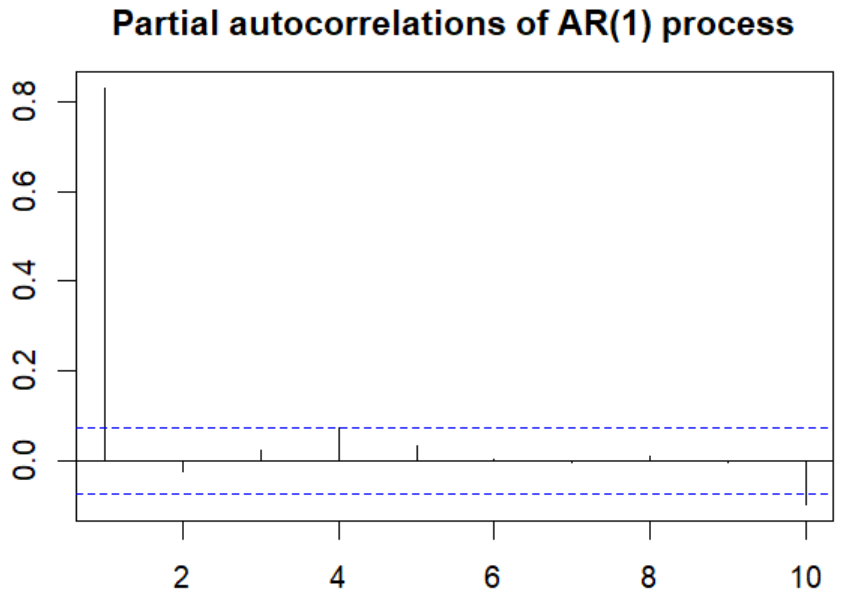
A linear combination of the time series and its own lag can be created, such that its lag 1 autocorrelation is zero.

The lag 2 autocorrelation of this new series is called the *partial autocorrelation* of lag 2, and represents the true second order autocorrelation.

The *partial autocorrelation* of lag `k` is the autocorrelation of lag `k`, after all the autocorrelations of lag `1, ..., k-1` have been removed.

The *partial autocorrelations* ϱ_i are the estimators of the coefficients ϕ_i of the *AR(n)* process.

The function `pacf()` calculates and plots the *partial autocorrelations* using the Durbin-Levinson algorithm.



An *AR(1)* process has an exponentially decaying *ACF* and a non-zero *PACF* at lag one.

```
> # PACF of AR(1) process
> pacfl <- pacf(arimav, lag=10, xlab="", ylab="", main="")
> title("Partial autocorrelations of AR(1) process", line=1)
> pacfl <- as.numeric(pacfl$acf)
> pacfl[1:5]
```

Stationary Processes and Unit Root Processes

A process is *stationary* if its probability distribution does not change with time, which means that it has constant mean and variance.

The *autoregressive* process *AR(n)*:
$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \ldots + \varphi_n r_{t-n} + \xi_t$

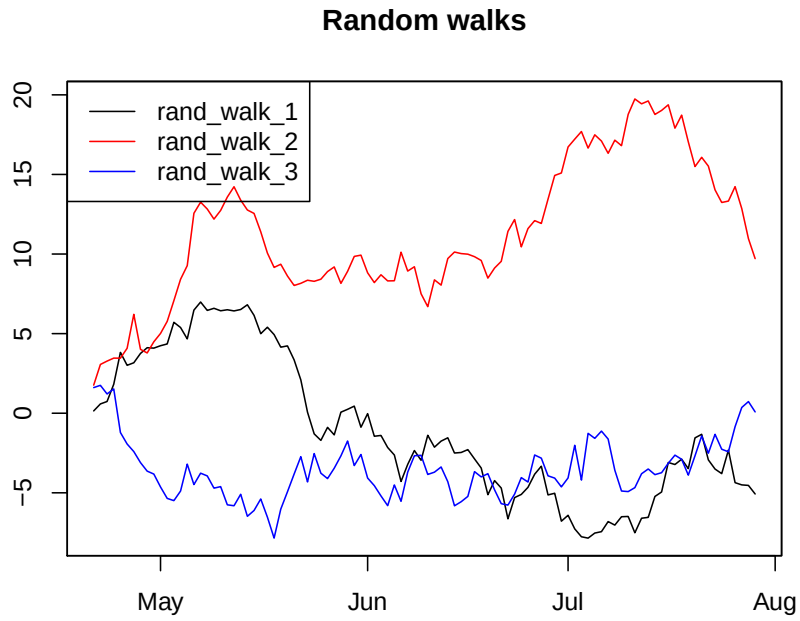
Has the following characteristic equation:
$1 - \varphi_1 z - \varphi_2 z^2 - \ldots - \varphi_n z^n = 0$

An autoregressive process is *stationary* only if the absolute values of all the roots of its characteristic equation are greater than 1.

If the sum of the autoregressive coefficients is equal to 1: $\sum_{i=1}^{n} \varphi_i = 1$, then the process has a root equal to 1 (it has a *unit root*), so it's not *stationary*.

Non-stationary processes with unit roots are called *unit root* processes.

A simple example of a *unit root* process is the *Brownian Motion*: $p_t = p_{t-1} + \xi_t$




```
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> randw <- cumsum(zoo(matrix(rnorm(3*100), ncol=3),
+ order.by=(Sys.Date()+0:99)))
> colnames(randw) <- paste("randw", 1:3, sep="_")
> plot.zoo(randw, main="Random walks",
+      xlab="", ylab="", plot.type="single",
+      col=c("black", "red", "blue"))
> # Add legend
> legend(x="topleft", legend=colnames(randw),
+  col=c("black", "red", "blue"), lty=1)
```

Integrated and Unit Root Processes

The cumulative sum of a given process is called its *integrated* process.

For example, asset prices follow an *integrated* process with respect to asset returns: $p_t = \sum_{i=1}^{t} r_i$.

If returns follow an *AR(n)* process:
$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \ldots + \varphi_n r_{t-n} + \xi_t$

Then asset prices follow the process:
$p_t = (1 + \varphi_1)p_{t-1} + (\varphi_2 - \varphi_1)p_{t-2} + \ldots + (\varphi_n - \varphi_{n-1})p_{t-n} - \varphi_n p_{t-n-1} + \xi_t$

The sum of the coefficients of the price process is equal to 1, so it has a *unit root* for all values of the φ_i coefficients.

The *integrated* process of an *AR(n)* process is always a *unit root* process.

For example, if returns follow an *AR(1)* process:
$r_t = \varphi r_{t-1} + \xi_t$.

Then asset prices follow the process:
$p_t = (1 + \varphi)p_{t-1} - \varphi p_{t-2} + \xi_t$

Which is a *unit root* process for all values of φ, because the sum of its coefficients is equal to 1.

If $\varphi = 0$ then the above process is a *Brownian Motion* (random walk).

```
> # Simulate arima with large AR coefficient
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nrows <- 1e4
> arimav <- arima.sim(n=nrows, model=list(ar=0.99))
> tseries::adf.test(arimav)
> # Integrated series has unit root
> tseries::adf.test(cumsum(arimav))
> # Simulate arima with negative AR coefficient
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> arimav <- arima.sim(n=nrows, model=list(ar=-0.99))
> tseries::adf.test(arimav)
> # Integrated series has unit root
> tseries::adf.test(cumsum(arimav))
```

The Variance of Unit Root Processes

An *AR(1)* process: $r_t = \varphi r_{t-1} + \xi_t$ has the following characteristic equation: $1 - \varphi z = 0$, with a root equal to: $z = 1/\varphi$

If $\varphi = 1$, then the characteristic equation has a *unit root* (and therefore it isn't *stationary*), and the process follows: $r_t = r_{t-1} + \xi_t$

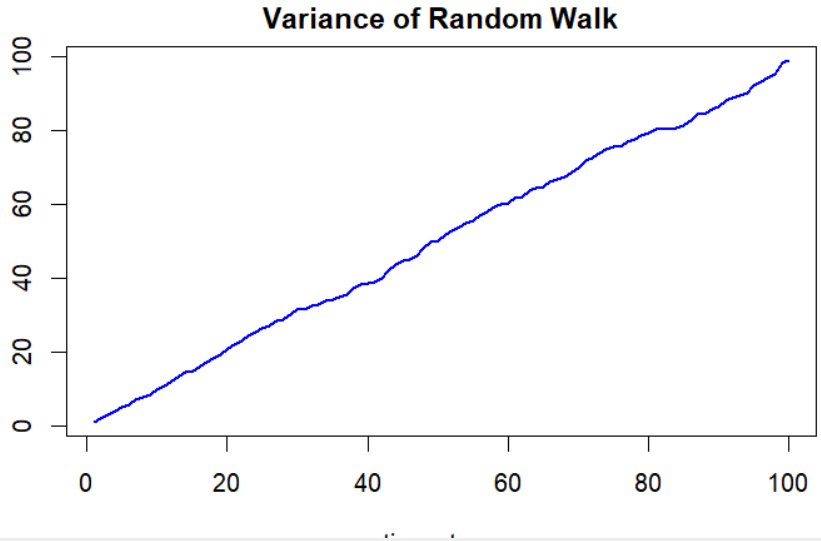
The above is called a *Brownian Motion*, and it's an example of a *unit root* process.

The expected value of the *AR(1)* process $r_t = \varphi r_{t-1} + \xi_t$ is equal to zero: $\mathbb{E}[r_t] = \frac{\mathbb{E}[\xi_t]}{1-\varphi} = 0$.

And its variance is equal to: $\sigma^2 = \mathbb{E}[r_i^2] = \frac{\sigma_\xi^2}{1-\varphi^2}$.

If $\varphi = 1$, then the *variance* grows over time and becomes infinite over time, so the process is not *stationary*.

The variance of the *Brownian Motion* $r_t = r_{t-1} + \xi$ is proportional to time: $\sigma_i^2 = \mathbb{E}[r_i^2] = i\sigma_\xi^2$




```
> # Simulate random walks using apply() loops
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> randws <- matrix(rnorm(1000*100), ncol=1000)
> randws <- apply(randws, 2, cumsum)
> varv <- apply(randws, 1, var)
> # Simulate random walks using vectorized functions
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> randws <- matrixStats::colCumsums(matrix(rnorm(1000*100), ncol=100
> varv <- matrixStats::rowVars(randws)
> par(mar=c(5, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot(varv, xlab="time steps", ylab="",
+      t="l", col="blue", lwd=2,
+      main="Variance of Random Walk")
```

The Brownian Motion Process

In the *Brownian Motion* process, the returns r_t are equal to the random *innovations*:

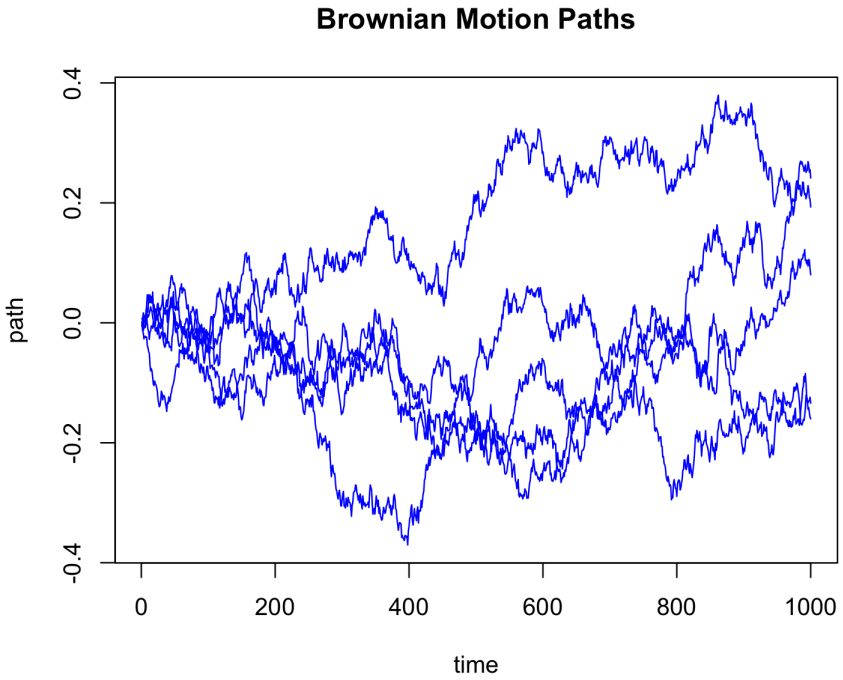
$$r_t = p_t - p_{t-1} = \sigma \xi_t$$

$$p_t = p_{t-1} + r_t$$

Where σ is the volatility of returns, and ξ_t are random normal *innovations* with zero mean and unit variance.

The *Brownian Motion* process for prices can be written as an *AR(1)* autoregressive process with coefficient $\varphi = 1$:

$$p_t = \varphi p_{t-1} + \sigma \xi_t$$




```
> # Define Brownian Motion parameters
> nrows <- 1000; sigmav <- 0.01
> # Simulate 5 paths of Brownian motion
> pricev <- matrix(rnorm(5*nrows, sd=sigmav), nc=5)
> pricev <- matrixStats::colCumsums(pricev)
> # Plot 5 paths of Brownian motion
> matplot(y=pricev, main="Brownian Motion Paths",
+   xlab="time", ylab="path",
+   type="l", lty="solid", lwd=1, col="blue")
> # Save plot to png file on Mac
> quartz.save("figure/brown_paths.png", type="png", width=6, height=4)
```

The Ornstein-Uhlenbeck Process

In the *Ornstein-Uhlenbeck* process, the returns r_t are equal to the difference between the equilibrium price μ minus the latest price p_{t-1}, times the mean reversion parameter θ, plus random *innovations*:

$$r_t = p_t - p_{t-1} = \theta\,(\mu - p_{t-1}) + \sigma\,\xi_t$$

$$p_t = p_{t-1} + r_t$$

Where σ is the volatility of returns, and ξ_t are random normal *innovations* with zero mean and unit variance.

The *Ornstein-Uhlenbeck* process for prices can be written as an *AR(1)* process plus a drift:

$$p_t = \theta\,\mu + (1 - \theta)\,p_{t-1} + \sigma\,\xi_t$$

The *Ornstein-Uhlenbeck* process cannot be simulated using the function `filter()` because of the drift term, so it must be simulated using explicit loops, either in `R` or in `C++`.

The compiled *Rcpp* `C++` code can be over 100 times faster than loops in `R`!

```
> # Define Ornstein-Uhlenbeck parameters
> prici <- 0.0; priceq <- 1.0;
> sigmav <- 0.02; thetav <- 0.01; nrows <- 1000
> # Initialize the data
> innov <- rnorm(nrows)
> retp <- numeric(nrows)
> pricev <- numeric(nrows)
> retp[1] <- sigmav*innov[1]
> pricev[1] <- prici
> # Simulate Ornstein-Uhlenbeck process in R
> for (i in 2:nrows) {
+   retp[i] <- thetav*(priceq - pricev[i-1]) + sigmav*innov[i]
+   pricev[i] <- pricev[i-1] + retp[i]
+ }  ##  end for
> # Simulate Ornstein-Uhlenbeck process in Rcpp
> pricecpp <- HighFreq::sim_ou(prici=prici, priceq=priceq,
+   theta=thetav, innov=matrix(sigmav*innov))
> all.equal(pricev, drop(pricecpp))
> # Compare the speed of R code with Rcpp
> library(microbenchmark)
> summary(microbenchmark(
+   Rcode={for (i in 2:nrows) {
+     retp[i] <- thetav*(priceq - pricev[i-1]) + sigmav*innov[i]
+     pricev[i] <- pricev[i-1] + retp[i]}},
+   Rcpp=HighFreq::sim_ou(prici=prici, priceq=priceq,
+     theta=thetav, innov=matrix(sigmav*innov)),
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

The Solution of the Ornstein-Uhlenbeck Process

The *Ornstein-Uhlenbeck* process in continuous time is:

$$\mathrm{d}p_t = \theta\,(\mu - p_t)\,\mathrm{d}t + \sigma\,\mathrm{d}B_t$$

Where B_t is a *Brownian Motion*, with $\mathrm{d}B_t$ following the normal distribution $\phi(0, \sqrt{\mathrm{d}t})$, with the volatility $\sqrt{\mathrm{d}t}$, equal to the square root of the time increment $\mathrm{d}t$.

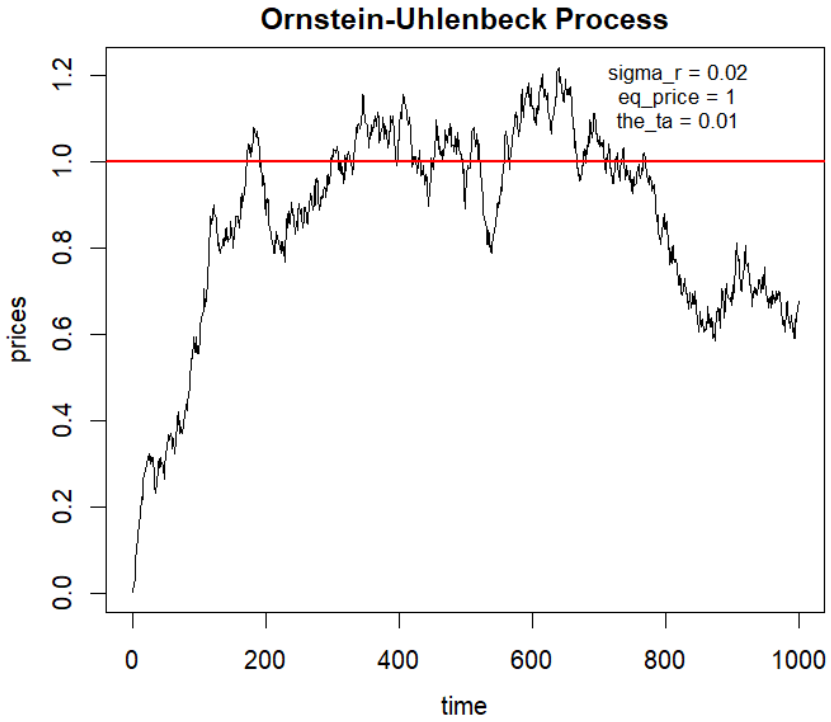
The solution of the *Ornstein-Uhlenbeck* process is given by:

$$p_t = p_0 e^{-\theta t} + \mu(1 - e^{-\theta t}) + \sigma \int_0^t e^{\theta(s-t)} \mathrm{d}W_s$$

The mean and variance are given by:
$\mathbb{E}[p_t] = p_0 e^{-\theta t} + \mu(1 - e^{-\theta t}) \to \mu$
$\mathbb{E}[(p_t - \mathbb{E}[p_t])^2] = \frac{\sigma^2}{2\theta}(1 - e^{-\theta t}) \to \frac{\sigma^2}{2\theta}$

The *Ornstein-Uhlenbeck* process is mean reverting to a non-zero equilibrium price μ.

The *Ornstein-Uhlenbeck* process needs a *warmup period* before it reaches equilibrium.

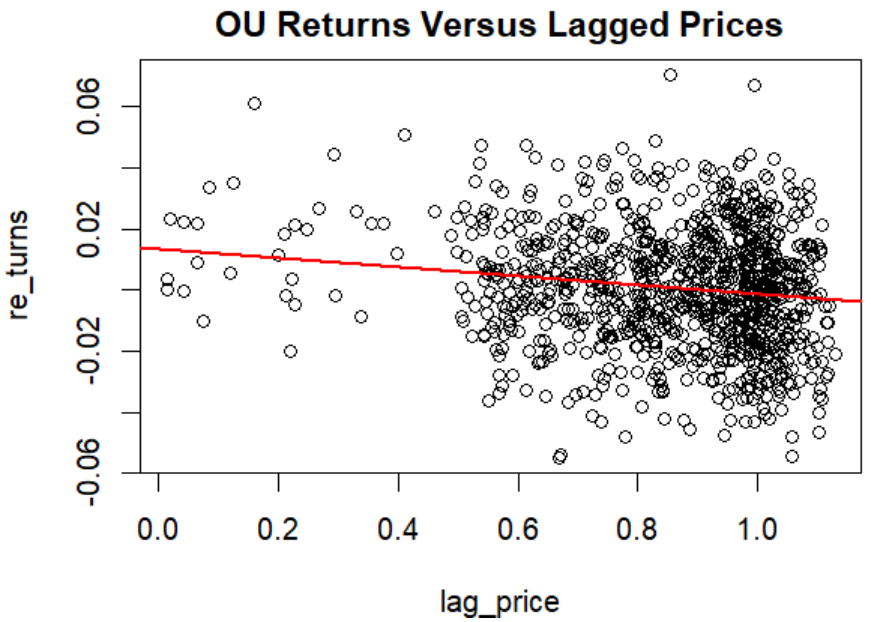



```
> plot(pricev, type="l", xlab="time", ylab="prices",
+      main="Ornstein-Uhlenbeck Process")
> legend("topright", title=paste(c(paste0("sigmav = ", sigmav),
+      paste0("priceq = ", ),
+      paste0("thetav = ", thetav)),
+    collapse="\n"),
+  legend="", cex=0.8, inset=0.1, bg="white", bty="n")
> abline(h=, col='red', lwd=2)
```

Ornstein-Uhlenbeck Process Returns Correlation

Under the *Ornstein-Uhlenbeck* process, the returns are negatively correlated to the lagged prices.

```
> retp <- rutils::diffit(pricev)
> pricelag <- rutils::lagit(pricev)
> formulav <- retp ~ pricelag
> regmod <- lm(formulav)
> summary(regmod)
> # Plot regression
> plot(formulav, main="OU Returns Versus Lagged Prices")
> abline(regmod, lwd=2, col="red")
```



Calibrating the Ornstein-Uhlenbeck Parameters

The volatility parameter of the Ornstein-Uhlenbeck process can be estimated directly from the standard deviation of the returns.

The θ and μ parameters can be estimated from the linear regression of the returns versus the lagged prices.

Calculating regression parameters directly from formulas has the advantage of much faster calculations.

```
> # Calculate volatility parameter
> c(volatility=sigmav, estimate=sd(retp))
> # Extract OU parameters from regression
> coeff <- summary(regmod)$coefficients
> # Calculate regression alpha and beta directly
> betac <- cov(retp, pricelag)/var(pricelag)
> alphac <- (mean(retp) - betac*mean(pricelag))
> cbind(direct=c(alpha=alphac, beta=betac), lm=coeff[, 1])
> all.equal(c(alpha=alphac, beta=betac), coeff[, 1],
+     check.attributes=FALSE)
> # Calculate regression standard errors directly
> betac <- c(alpha=alphac, beta=betac)
> fitv <- (alphac + betac*pricelag)
> resids <- (retp - fitv)
> price2 <- sum((pricelag - mean(pricelag))^2)
> betasd <- sqrt(sum(resids^2)/price2/(nrows-2))
> alphasd <- sqrt(sum(resids^2)/(nrows-2)*(1:nrows + mean(pricelag)
> cbind(direct=c(alphasd=alphasd, betasd=betasd), lm=coeff[, 2])
> all.equal(c(alphasd=alphasd, betasd=betasd), coeff[, 2],
+     check.attributes=FALSE)
> # Compare mean reversion parameter theta
> c(theta=(-thetav), round(coeff[2, ], 3))
> # Compare equilibrium price mu
> c(priceq=priceq, estimate=-coeff[1, 1]/coeff[2, 1])
> # Compare actual and estimated parameters
> coeff <- cbind(c(thetav*priceq, -thetav), coeff[, 1:2])
> rownames(coeff) <- c("drift", "theta")
> colnames(coeff)[1] <- "actual"
> round(coeff, 4)
```

The Schwartz Process

The *Ornstein-Uhlenbeck* prices can be negative, while actual prices are usually not negative.

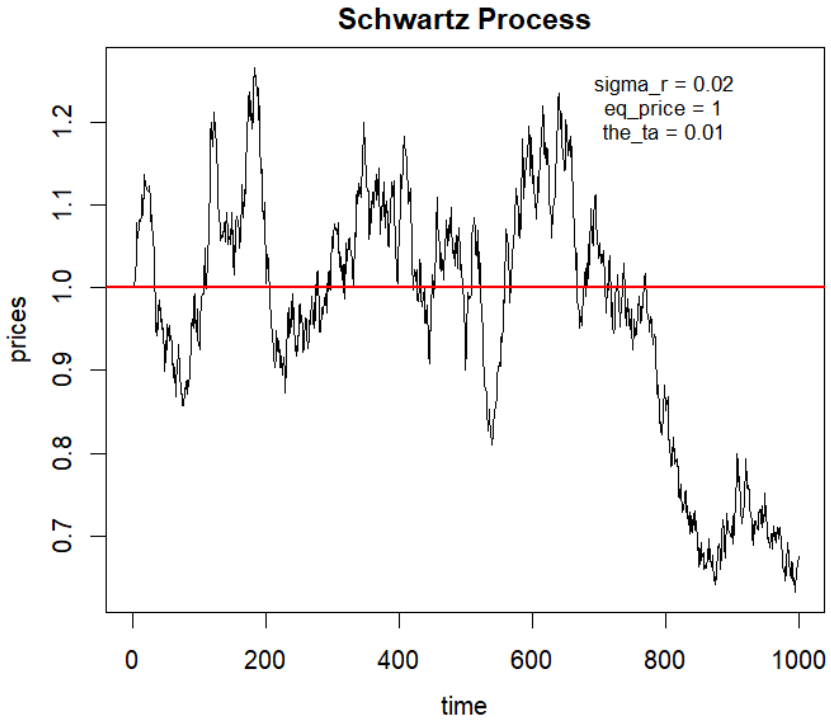
So the *Ornstein-Uhlenbeck* process is better suited for simulating the logarithm of prices, which can be negative.

The *Schwartz* process is the exponential of the *Ornstein-Uhlenbeck* process, so it avoids negative prices by compounding the percentage returns r_t instead of summing them:

$$r_t = \log p_t - \log p_{t-1} = \theta (\mu - p_{t-1}) + \sigma \xi_t$$

$$p_t = p_{t-1} \exp(r_t)$$

Where the parameter θ is the strength of mean reversion, σ is the volatility, and ξ_t are random normal *innovations* with zero mean and unit variance.




```
> # Simulate Schwartz process
> retp <- numeric(nrows)
> pricev <- numeric(nrows)
> pricev[1] <- exp(sigmav*innov[1])
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")  ##
> for (i in 2:nrows) {
+   retp[i] <- thetav*(priceq - pricev[i-1]) + sigmav*innov[i]
+   pricev[i] <- pricev[i-1]*exp(retp[i])
+ }  ##  end for
```

```
> plot(pricev, type="l", xlab="time", ylab="prices",
+      main="Schwartz Process")
> legend("topright",
+  title=paste(c(paste0("sigmav = ", sigmav),
+      paste0("priceq = ", priceq),
+      paste0("thetav = ", thetav)),
+    collapse="\n"),
+  legend="", cex=0.8, inset=0.12, bg="white", bty="n")
> abline(h=priceq, col='red', lwd=2)
```

The Dickey-Fuller Process

The *Dickey-Fuller* process is a combination of an *Ornstein-Uhlenbeck* process and an *autoregressive* process.

The returns r_t are equal to the sum of a mean reverting term plus *autoregressive* terms:

$$r_t = \theta(\mu - p_{t-1}) + \varphi_1 r_{t-1} + \ldots + \varphi_n r_{t-n} + \sigma \xi_t$$

$$p_t = p_{t-1} + r_t$$

Where μ is the equilibrium price, σ is the volatility of returns, θ is the strength of mean reversion, and ξ_t are standard normal *innovations*.

Then the prices follow an *autoregressive* process:

$$p_t = \theta\mu + (1 + \varphi_1 - \theta)p_{t-1} + (\varphi_2 - \varphi_1)p_{t-2} + \ldots + (\varphi_n - \varphi_{n-1})p_{t-n} - \varphi_n p_{t-n-1} + \sigma \xi_t$$

The sum of the *autoregressive* coefficients is equal to $1 - \theta$, so if the mean reversion parameter θ is positive: $\theta > 0$, then the time series p_t exhibits mean reversion and has no *unit root*.

```
> # Define Dickey-Fuller parameters
> prici <- 0.0;  priceq <- 1.0
> thetav <- 0.01;  nrows <- 1000
> coeff <- c(0.1, 0.39, 0.5)
> # Initialize the data
> innov <- rnorm(nrows, sd=0.01)
> retp <- numeric(nrows)
> pricev <- numeric(nrows)
> # Simulate Dickey-Fuller process using recursive loop in R
> retp[1] <- innov[1]
> pricev[1] <- prici
> retp[2] <- thetav*(priceq - pricev[1]) + coeff[1]*retp[1] +
+   innov[2]
> pricev[2] <- pricev[1] + retp[2]
> retp[3] <- thetav*(priceq - pricev[2]) + coeff[1]*retp[2] +
+   coeff[2]*retp[1] + innov[3]
> pricev[3] <- pricev[2] + retp[3]
> for (it in 4:nrows) {
+   retp[it] <- thetav*(priceq - pricev[it-1]) +
+     retp[(it-1):(it-3)] %*% coeff + innov[it]
+   pricev[it] <- pricev[it-1] + retp[it]
+ }  ##  end for
> # Simulate Dickey-Fuller process in Rcpp
> pricecpp <- HighFreq::sim_df(prici=prici, priceq=priceq,
+    theta=thetav, coeff=matrix(coeff), innov=matrix(innov))
> # Compare prices
> all.equal(pricev, drop(pricecpp))
> # Compare the speed of R code with Rcpp
> library(microbenchmark)
> summary(microbenchmark(
+   Rcode={for (it in 4:nrows) {
+   retp[it] <- thetav*(priceq - pricev[it-1]) + retp[(it-1):(it-3)]
+   pricev[it] <- pricev[it-1] + retp[it]
+   }},
+   Rcpp=HighFreq::sim_df(prici=prici, priceq=priceq, theta=thetav,
+   times=10))[, c(1, 4, 5)]  ##  end microbenchmark summary
```

Augmented Dickey-Fuller ADF Test for Unit Roots

The *Augmented Dickey-Fuller ADF* test is designed to test the *null hypothesis* that a time series has a *unit root*.

The *ADF* test fits an autoregressive model for the prices p_t:

$$r_t = \theta(\mu - p_{t-1}) + \varphi_1 r_{t-1} + \ldots + \varphi_n r_{t-n} + \sigma \xi_t$$

$$p_t = p_{t-1} + r_t$$

Where μ is the equilibrium price, σ is the volatility of returns, and θ is the strength of mean reversion.

ε_i are the *residuals*, which are assumed to be standard normally distributed $\phi(0, \sigma_\varepsilon)$, independent, and stationary.

If the mean reversion parameter θ is positive: $\theta > 0$, then the time series p_t exhibits mean reversion and has no *unit root*.

The *null hypothesis* is that prices have a unit root ($\theta = 0$, no mean reversion), while the alternative hypothesis is that it's *stationary* ($\theta > 0$, mean reversion).

The *ADF* test statistic is equal to the *t*-value of the θ parameter: $t_\theta = \hat{\theta}/SE_\theta$ (which follows a different distribution from the t-distribution).

The function `tseries::adf.test()` performs the *ADF* test.

```
> # Simulate AR(1) process with coefficient=1, with unit root
> innov <- matrix(rnorm(1e4, sd=0.01))
> arimav <- HighFreq::sim_ar(coeff=matrix(1), innov=innov)
> plot(arimav, t="l", main="Brownian Motion")
> # Perform ADF test with lag = 1
> tseries::adf.test(arimav, k=1)
> # Perform standard Dickey-Fuller test
> tseries::adf.test(arimav, k=0)
> # Simulate AR(1) with coefficient close to 1, without unit root
> arimav <- HighFreq::sim_ar(coeff=matrix(0.99), innov=innov)
> plot(arimav, t="l", main="AR(1) coefficient = 0.99")
> tseries::adf.test(arimav, k=1)
> # Simulate Ornstein-Uhlenbeck OU process with mean reversion
> prici <- 0.0; priceq <- 0.0; thetav <- 0.1
> pricev <- HighFreq::sim_ou(prici=prici, priceq=priceq,
+   theta=thetav, innov=innov)
> plot(pricev, t="l", main=paste("OU coefficient =", thetav))
> tseries::adf.test(pricev, k=1)
> # Simulate Ornstein-Uhlenbeck OU process with zero reversion
> thetav <- 0.0
> pricev <- HighFreq::sim_ou(prici=prici, priceq=priceq,
+   theta=thetav, innov=innov)
> plot(pricev, t="l", main=paste("OU coefficient =", thetav))
> tseries::adf.test(pricev, k=1)
```

The common practice is to use a small number of lags in the *ADF* test, and if the residuals are autocorrelated, then to increase them until the correlations are no longer significant.

If the number of lags in the regression is zero: $n = 0$ then the *ADF* test becomes the standard *Dickey-Fuller* test: $r_t = \theta(\mu - p_{t-1}) + \varepsilon_i$.

Sensitivity of the ADF Test for Detecting Unit Roots

The *ADF null hypothesis* is that prices have a unit root, while the alternative hypothesis is that they're *stationary*.

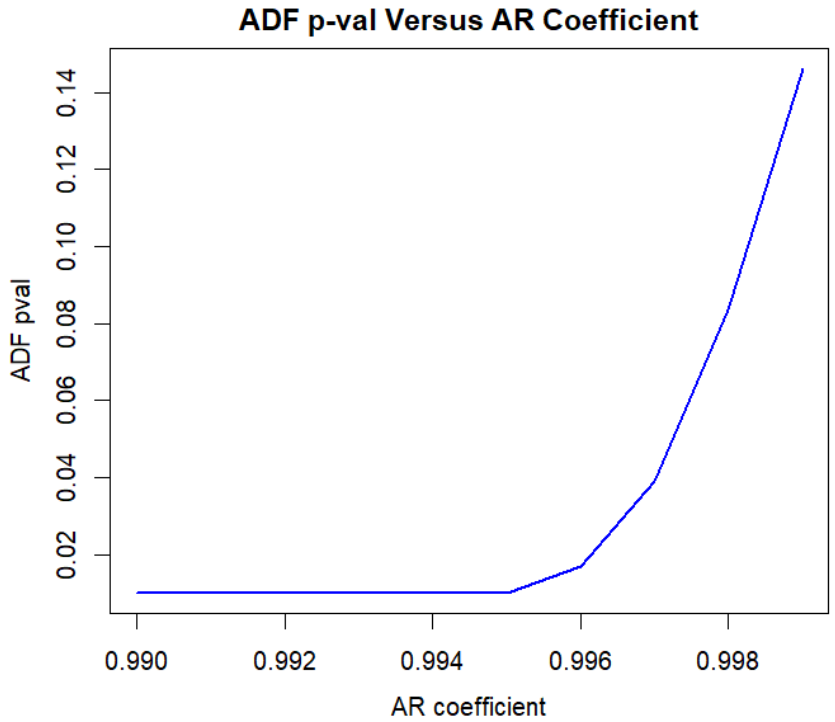
The *ADF* test has low *sensitivity*, i.e. the ability to correctly identify time series with no *unit root*, causing it to produce *false negatives* (*type II* errors).

This is especially true for time series which exhibit mean reversion over longer time horizons. The *ADF* test will identify them as having a *unit root* even though they are mean reverting.

Therefore the *ADF* test often requires a lot of data before it's able to correctly identify *stationary* time series with *no unit root*.

A *true negative* test result is that the *null hypothesis* is `TRUE` (pricev have a unit root), while a *true positive* result is that the *null hypothesis* is `FALSE` (pricev are stationary).

The function `tseries::adf.test()` assumes that the data is *normally distributed*, which may underestimate the standard errors of the parameters, and produce *false positives* (*type I* errors) by incorrectly rejecting the null hypothesis of a unit root process.




```
> # Simulate AR(1) process with different coefficients
> coeffv <- seq(0.99, 0.999, 0.001)
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> adft <- sapply(coeffv, function(coeff) {
+   arimav <- filter(x=retp, filter=coeff, method="recursive")
+   adft <- suppressWarnings(tseries::adf.test(arimav))
+   c(adfstat=unname(adft$statistic), pval=adft$p.value)
+ })  ##  end sapply
> dev.new(width=6, height=4, noRStudioGD=TRUE)
> # x11(width=6, height=4)
> plot(x=coeffv, y=adft["pval", ], main="ADF p-val Versus AR Coeffic
+      xlab="AR coefficient", ylab="ADF pval", t="l", col="blue", lw
> plot(x=coeffv, y=adft["adfstat", ], main="ADF Stat Versus AR Coeff
+      xlab="AR coefficient", ylab="ADF stat", t="l", col="blue", lw
```

Fitting Time Series to Autoregressive Models

An *autoregressive* process *AR(n)* for the time series of returns r_t:

$$r_t = \varphi_1 r_{t-1} + \varphi_2 r_{t-2} + \ldots + \varphi_n r_{t-n} + \xi_t = \sum_{j=1}^{n} \varphi_j r_{t-j} + \xi_t$$

The coefficients φ can be calculated using linear regression, with the *response* equal to $\mathbf{r}$, and the columns of the *predictor matrix* $\mathbb{P}$ equal to the lags of $\mathbf{r}$:

$$\varphi = \mathbb{P}^{-1}\mathbf{r}$$

An intercept term can be added to the above formula by adding a unit column to the predictor matrix $\mathbb{P}$.

Adding the intercept term produces slightly different coefficients, depending on the mean of the returns.

The function `HighFreq::sim_ar()` simulates an *AR(n)* processes using `C++` code.

The function `stats::ar.ols()` fits an *AR(n)* model, but it produces slightly different coefficients than linear regression, because it uses a different calibration procedure.

```
> # Specify AR process parameters
> nrows <- 1e3
> coeff <- matrix(c(0.1, 0.39, 0.5)); ncoeff <- NROW(coeff)
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection"); innov
> # Simulate AR process using HighFreq::sim_ar()
> arimav <- HighFreq::sim_ar(coeff=coeff, innov=innov)
> # Fit AR model using ar.ols()
> arfit <- ar.ols(arimav, order.max=ncoeff, aic=FALSE)
> class(arfit)
> is.list(arfit)
> drop(arfit$ar); drop(coeff)
> # Define predictor matrix without intercept column
> predm <- sapply(1:ncoeff, rutils::lagit, input=arimav)
> # Fit AR model using regression
> predinv <- MASS::ginv(predm)
> coeff <- drop(predinv %*% arimav)
> all.equal(drop(arfit$ar), coeff, check.attributes=FALSE)
```

The Standard Errors of the *AR(n)* Coefficients

The *standard errors* of the fitted *AR(n)* coefficients are proportional to the standard deviation of the fitted residuals.

Their *t*-values are equal to the ratio of the fitted coefficients divided by their standard errors.

```
> # Calculate the model residuals
> fitv <- drop(predm %*% coeff)
> resids <- drop(arimav - fitv)
> # Variance of residuals
> residsd <- sum(resids^2)/(nrows-NROW(coeff))
> # Inverse of predictor matrix squared
> pred2 <- MASS::ginv(crossprod(predm))
> # Calculate covariance matrix of AR coefficients
> covmat <- residsd*pred2
> coefsd <- sqrt(diag(covmat))
> # Calculate t-values of AR coefficients
> coefft <- drop(coeff)/coefsd
> # Plot the t-values of the AR coefficients
> barplot(coefft, xlab="lag", ylab="t-value",
+   main="Coefficient t-values of AR Forecasting Model")
```

Order Selection of *AR(n)* Model

Order selection means determining the *order parameter* n of the *AR(n)* model that best fits the time series.

The order parameter n can be set equal to the number of significantly non-zero *partial autocorrelations* of the time series.

The order parameter can also be determined by only selecting coefficients with statistically significant t-values.

Fitting an *AR(n)* model can be performed by first determining the order n, and then calculating the coefficients.

The function `stats::arima()` calibrates (fits) an *ARIMA* model to a univariate time series.

The function `auto.arima()` from the package *forecast* performs order selection, and calibrates an *AR(n)* model to a univariate time series.

```
> # Fit AR(5) model into AR(3) process
> predm <- sapply(1:5, rutils::lagit, input=arimav)
> predinv <- MASS::ginv(predm)
> coeff <- drop(predinv %*% arimav)
> # Calculate t-values of AR(5) coefficients
> resids <- drop(arimav - drop(predm %*% coeff))
> residsd <- sum(resids^2)/(nrows-NROW(coeff))
> covmat <- residsd*MASS::ginv(crossprod(predm))
> coefsd <- sqrt(diag(covmat))
> coefft <- drop(coeff)/coefsd
> # Fit AR(5) model using arima()
> arfit <- arima(arimav, order=c(5, 0, 0), include.mean=FALSE)
> arfit$coef
> # Fit AR(5) model using auto.arima()
> library(forecast)  ##  Load forecast
> arfit <- forecast::auto.arima(arimav, max.p=5, max.q=0, max.d=0)
> # Fit AR(5) model into VTI returns
> retp <- drop(zoo::coredata(na.omit(rutils::etfenv$returns$VTI)))
> predm <- sapply(1:5, rutils::lagit, input=retp)
> predinv <- MASS::ginv(predm)
> coeff <- drop(predinv %*% retp)
> # Calculate t-values of AR(5) coefficients
> resids <- drop(retp - drop(predm %*% coeff))
> residsd <- sum(resids^2)/(nrows-NROW(coeff))
> covmat <- residsd*MASS::ginv(crossprod(predm))
> coefsd <- sqrt(diag(covmat))
> coefft <- drop(coeff)/coefsd
```

The Yule-Walker Equations

The Yule-Walker equations relate the *autocorrelation coefficients* ρ_i with the coefficients of the *AR(n)* process φ_i.

To lighten the notation we can assume that the time series r_t has zero mean $\mathbb{E}[r_t] = 0$ and unit variance $\mathbb{E}[r_i^2] = 1$. ($\mathbb{E}$ is the expectation operator.)

Then the *autocorrelations* of r_t are equal to: $\rho_k = \mathbb{E}[r_t r_{t-k}]$.

If we multiply the *autoregressive* process *AR(n)*: $r_t = \sum_{j=1}^{n} \varphi_j r_{t-j} + \xi_t$, by r_{t-k} and take the expectations, then we obtain the Yule-Walker equations:

$$\begin{pmatrix} \rho_1 \\ \rho_2 \\ \rho_3 \\ \vdots \\ \rho_n \end{pmatrix} = \begin{pmatrix} 1 & \rho_1 & \cdots & \rho_{n-1} \\ \rho_1 & 1 & \cdots & \rho_{n-2} \\ \rho_2 & \rho_1 & \cdots & \rho_{n-3} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{n-1} & \rho_{n-2} & \cdots & 1 \end{pmatrix} \begin{pmatrix} \varphi_1 \\ \varphi_2 \\ \varphi_3 \\ \vdots \\ \varphi_n \end{pmatrix}$$

The Yule-Walker equations can be solved for the *AR(n)* coefficients φ_i using matrix inversion.

```
> # Compute autocorrelation coefficients
> acfl <- rutils::plot_acf(arimav, lag=10, plot=FALSE)
> acfl <- drop(acfl$acf)
> nrows <- NROW(acfl)
> acf1 <- c(1, acfl[-nrows])
> # Define Yule-Walker matrix
> ywmat <- sapply(1:nrows, function(lagg) {
+   if (lagg < nrows)
+     c(acf1[lagg:1], acf1[2:(nrows-lagg+1)])
+   else
+     acf1[lagg:1]
+ })  ##  end sapply
> # Generalized inverse of Yule-Walker matrix
> ywmatinv <- MASS::ginv(ywmat)
> # Solve Yule-Walker equations
> ywcoeff <- drop(ywmatinv %*% acfl)
> round(ywcoeff, 5)
> coeff
```

The Durbin-Levinson Algorithm for Partial Autocorrelations

The *partial autocorrelations* ϱ_i are the estimators of the coefficients φ_i of the *AR(n)* process.

The *partial autocorrelations* ϱ_i can be calculated by inverting the Yule-Walker equations.

The *partial autocorrelations* ϱ_i of an *AR(n)* process can be computed recursively from the autocorrelations ρ_i using the Durbin-Levinson algorithm:

$$\varrho_{i,i} = \frac{\rho_i - \sum_{k=1}^{i-1} \varrho_{i-1,k}\rho_{i-k}}{1 - \sum_{k=1}^{i-1} \varrho_{i-1,k}\rho_k}$$

$$\varrho_{i,k} = \varrho_{i-1,k} - \varrho_{i,i}\varrho_{i-1,i-k} \quad (1 \leq k \leq (i-1))$$

The diagonal elements $\varrho_{i,i}$ are updated first using the first equation. Then the off-diagonal elements $\varrho_{i,k}$ are updated using the second equation.

The *partial autocorrelations* are the diagonal elements: $\varrho_i = \varrho_{i,i}$

The Durbin-Levinson algorithm solves the Yule-Walker equations efficiently, without matrix inversion.

The function `pacf()` calculates and plots the *partial autocorrelations* using the Durbin-Levinson algorithm.

```
> # Calculate PACF from acf using Durbin-Levinson algorithm
> acfl <- rutils::plot_acf(arimav, lag=10, plotobj=FALSE)
> acfl <- drop(acfl$acf)
> nrows <- NROW(acfl)
> pacfl <- numeric(2)
> pacfl[1] <- acfl[1]
> pacfl[2] <- (acfl[2] - acfl[1]^2)/(1 - acfl[1]^2)
> # Calculate PACF recursively in a loop using Durbin-Levinson algor
> pacfll <- matrix(numeric(nrows*nrows), nc=nrows)
> pacfll[1, 1] <- acfl[1]
> for (it in 2:nrows) {
+   pacfll[it, it] <- (acfl[it] - pacfll[it-1, 1:(it-1)] %*% acfl[(i
+   for (it2 in 1:(it-1)) {
+     pacfll[it, it2] <- pacfll[it-1, it2] - pacfll[it, it] %*% pacf
+   }  ##  end for
+ }  ##  end for
> pacfll <- diag(pacfll)
> # Compare with the PACF without loop
> all.equal(pacfl, pacfll[1:2])
> # Calculate PACF using pacf()
> pacfl <- pacf(arimav, lag=10, plot=FALSE)
> pacfl <- drop(pacfl$acf)
> all.equal(pacfl, pacfll)
```

Downloading Stock Time Series From *Tiingo*

Yahoo data quality deteriorated significantly in 2017, and *Google* data quality is also poor, leaving *Tiingo*, *Alpha Vantage*, and Quandl as the only major providers of free daily *OHLC* stock prices.

But Quandl doesn't provide free *ETF* prices, while *Tiingo* does.

The function `getSymbols()` has a *method* for downloading time series data from *Tiingo*, called `getSymbols.tiingo()`.

Users must first obtain a *Tiingo API key*, and then pass it in `getSymbols.tiingo()` calls: https://www.tiingo.com/

Note that the data are downloaded as `xts` time series, with a date-time index of class `POSIXct` (not `Date`).

```
> # Load package rutils
> library(rutils)
> # Create new environment for data
> sp500env <- new.env()
> # Boolean vector of symbols already downloaded
> isdown <- symbolv %in% ls(sp500env)
> # Download in while loop from Tiingo and copy into environment
> nattempts <- 0  ##  Number of download attempts
> while ((sum(!isdown) > 0) & (nattempts<3)) {
+   ##  Download data and copy it into environment
+   nattempts <- nattempts + 1
+   cat("Download attempt = ", nattempts, "\n")
+   for (symboln in symbolv[!isdown]) {
+     cat("processing: ", symboln, "\n")
+     tryCatch(  ##  With error handler
+ quantmod::getSymbols(symboln, src="tiingo", adjust=TRUE, auto.assi
+            from="1990-01-01", env=sp500env, api.key="j84ac2b9c5bde
+ ##  Error handler captures error condition
+ error=function(msg) {
+   print(paste0("Error handler: ", msg))
+ },  ##  end error handler
+ finally=print(paste0("Symbol = ", symboln))
+     )  ##  end tryCatch
+   }  ##  end for
+   ##  Update vector of symbols already downloaded
+   isdown <- symbolv %in% ls(sp500env)
+   Sys.sleep(2)  ##  Wait 2 seconds until next attempt
+ }  ##  end while
> class(sp500env$AAPL)
> class(zoo::index(sp500env$AAPL))
> tail(sp500env$AAPL)
> symbolv[!isdown]
```

Coercing Date-time Indices

The date-time indices of the *OHLC* stock prices are in the `POSIXct` format suitable for intraday prices, not daily prices.

The function `as.Date()` coerces `POSIXct` objects into `Date` objects.

The function `get()` retrieves objects that are referenced using character strings, instead of their names.

The function `assign()` assigns a value to an object in a specified *environment*, by referencing it using a character string (name).

The functions `get()` and `assign()` allow retrieving and assigning values to objects that are referenced using character strings.

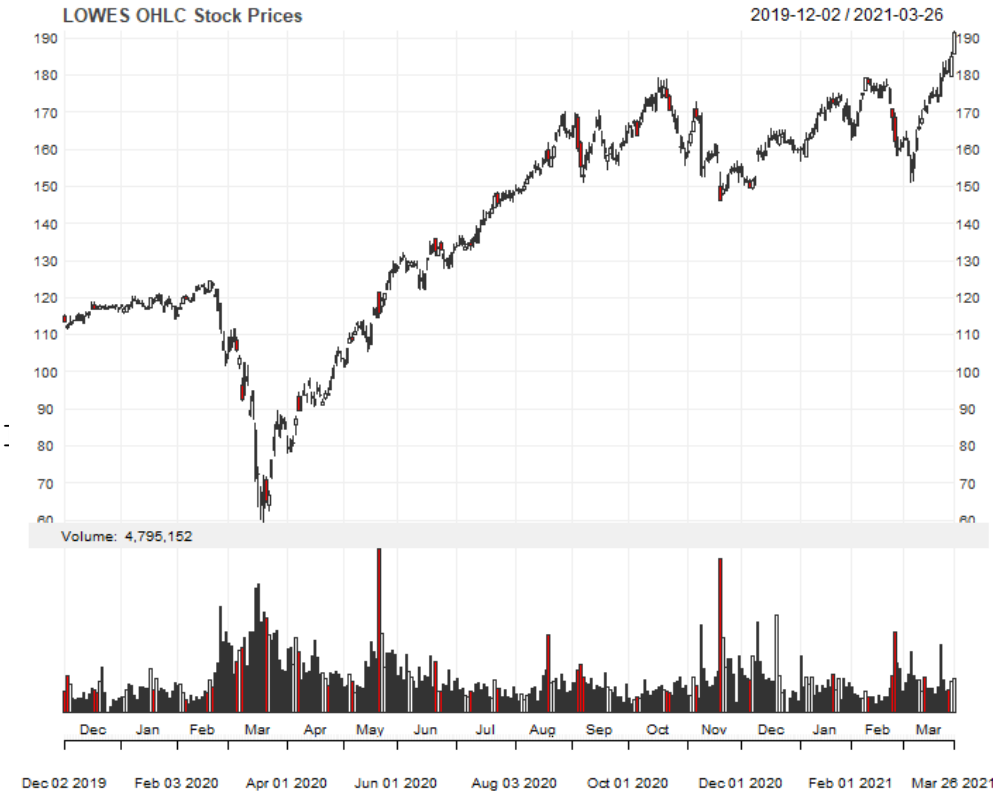
```
> # The date-time index of AAPL is POSIXct
> class(zoo::index(sp500env$AAPL))
> # Coerce the date-time index of AAPL to Date
> zoo::index(sp500env$AAPL) <- as.Date(zoo::index(sp500env$AAPL))
> # Coerce all the date-time indices to Date
> for (symboln in ls(sp500env)) {
+   ohlc <- get(symboln, envir=sp500env)
+   zoo::index(ohlc) <- as.Date(zoo::index(ohlc))
+   assign(symboln, ohlc, envir=sp500env)
+ }  ##  end for
```

Managing Exceptions in Stock Symbols

The column names for symbol "LOW" (Lowe's company) must be renamed for the extractor function quantmod::Lo() to work properly.

Tickers which contain a dot in their name (like "BRK.B") are not valid symbols in R, so they must be downloaded separately and renamed.

```
> # "LOW.Low" is a bad column name
> colnames(sp500env$LOW)
> strsplit(colnames(sp500env$LOW), split="[.]")
> do.call(cbind, strsplit(colnames(sp500env$LOW), split="[.]"))
> do.call(cbind, strsplit(colnames(sp500env$LOW), split="[.]"))[2, ]
> # Extract proper names from column names
> namev <- rutils::get_name(colnames(sp500env$LOW), field=2)
> # Or
> # namev <- do.call(rbind, strsplit(colnames(sp500env$LOW),
> #                                  split="[.]"))[, 2]
> # Rename "LOW" colnames to "LOWES"
> colnames(sp500env$LOW) <- paste("LOWES", namev, sep=".")
> sp500env$LOWES <- sp500env$LOW
> rm(LOW, envir=sp500env)
> # Rename BF-B colnames to "BFB"
> colnames(sp500env$"BF-B") <- paste("BFB", namev, sep=".")
> sp500env$BFB <- sp500env$"BF-B"
> rm("BF-B", envir=sp500env)
> # Rename BRK-B colnames
> sp500env$BRKB <- sp500env$`BRK-B`
> rm(`BRK-B`, envir=sp500env)
> colnames(sp500env$BRKB) <- gsub("BRK-B", "BRKB", colnames(sp500e
> # Save OHLC prices to .RData file
> save(sp500env, file="/Users/jerzy/Develop/lecture_slides/data/sp500.RData")
> # Download "BRK.B" separately with auto.assign=FALSE
> # BRKB <- quantmod::getSymbols("BRK-B", auto.assign=FALSE, src="tiingo", adjust=TRUE, from="1990-01-01", api.key="j84ac2b9c5bde2d68e3
> # colnames(BRKB) <- paste("BRKB", namev, sep=".")
> # sp500env$BRKB <- BRKB
```



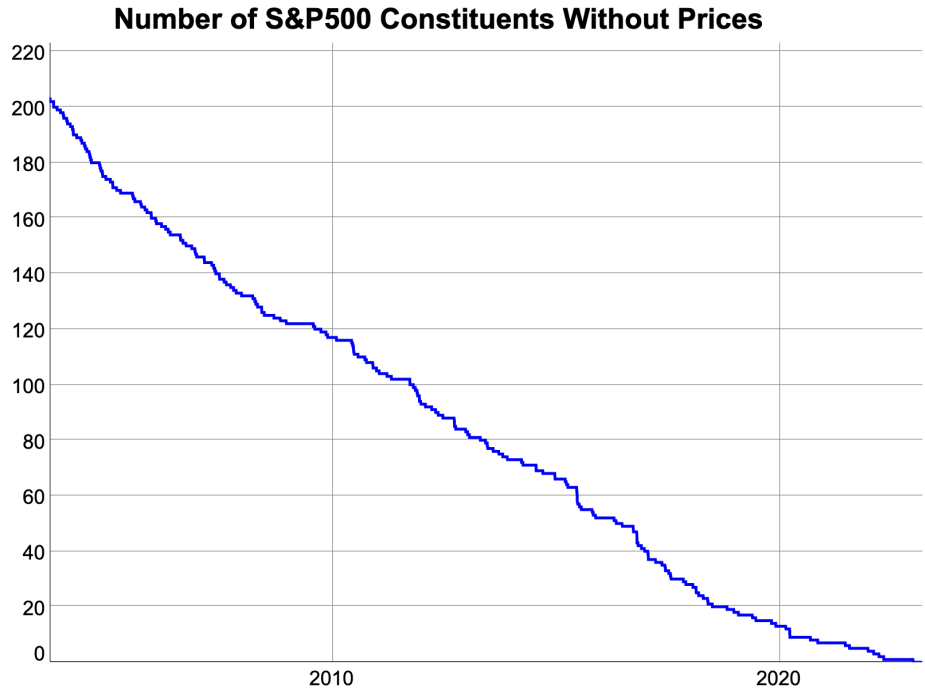

```
> # Plot OHLC candlestick chart for LOWES
> chart_Series(x=sp500env$LOWES["2019-12/"],
+   TA="add_Vo()", name="LOWES OHLC Stock Prices")
> # Plot dygraph
> dygraphs::dygraph(sp500env$LOWES["2019-12/", -5], main="LOWES OHL
+   dyCandlestick()
```

S&P500 Stock Index Constituent Prices

The file sp500.RData contains the *environment* sp500_env with *OHLC* prices and trading volumes of *S&P500* stock index constituents.

The *S&P500* stock index constituent data is of poor quality before 2000, so we'll mostly use the data after the year 2000.

```
> # Load S&P500 constituent stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500.RData")
> pricev <- eapply(sp500env, quantmod::Cl)
> pricev <- rutils::do_call(cbind, pricev)
> # Carry forward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> # Drop ".Close" from column names
> colnames(pricev)
> colnames(pricev) <- rutils::get_name(colnames(pricev))
> # Or
> # colnames(pricev) <- do.call(rbind,
> #   strsplit(colnames(pricev), split="[.]"))[, 1]
> # Calculate percentage returns of the S&P500 constituent stocks
> # retp <- xts::diff.xts(log(pricev))
> retp <- xts::diff.xts(pricev)/rutils::lagit(pricev, pad_zeros=FALSE)
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> samplev <- sample(NCOL(retp), s=100, replace=FALSE)
> prices100 <- pricev[, samplev]
> returns100 <- retp[, samplev]
> save(pricev, prices100,
+   file="/Users/jerzy/Develop/lecture_slides/data/sp500_prices.RData")
> save(retp, returns100,
+   file="/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RData")
```

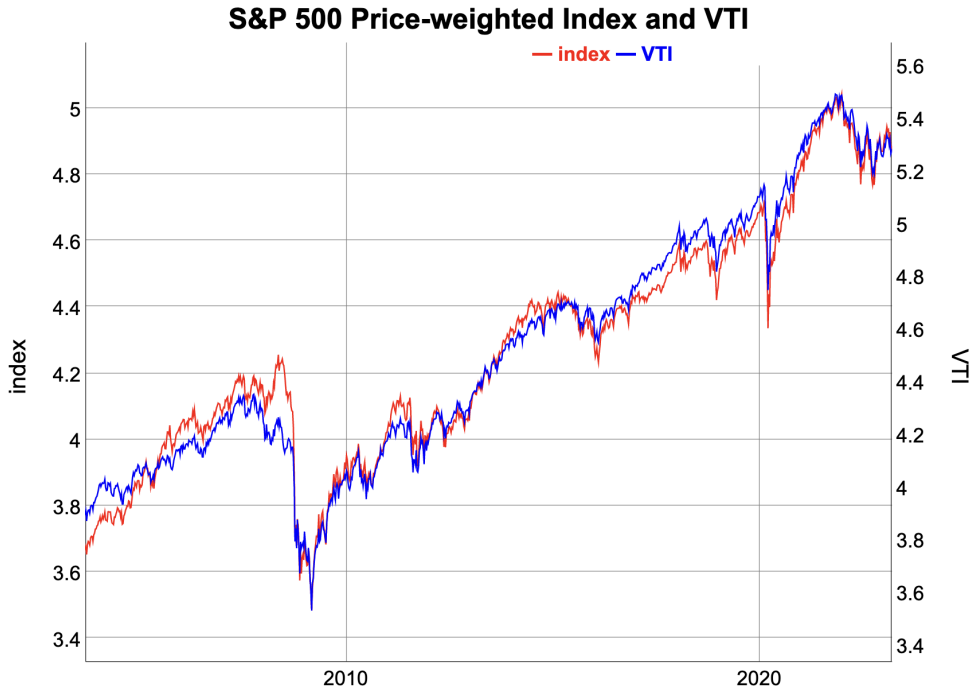



```
> # Calculate number of constituents without prices
> datav <- rowSums(is.na(pricev))
> datav <- xts::xts(datav, order.by=zoo::index(pricev))
> dygraphs::dygraph(datav, main="Number of S&P500 Constituents Witho
+   dyOptions(colors="blue", strokeWidth=2)
```

S&P500 Stock Portfolio Index

The price-weighted index of *S&P500* constituents closely follows the VTI *ETF*.

```
> # Calculate price weighted index of constituent
> ncols <- NCOL(pricev)
> pricev <- zoo::na.locf(pricev, fromLast=TRUE)
> indeks <- xts(rowSums(pricev)/ncols, zoo::index(pricev))
> colnames(indeks) <- "index"
> # Combine index with VTI
> datav <- cbind(indeks[zoo::index(etfenv$VTI)], etfenv$VTI[, 4])
> colv <- c("index", "VTI")
> colnames(datav) <- colv
> # Plot index with VTI
> endw <- rutils::calc_endpoints(datav, interval="weeks")
> dygraphs::dygraph(log(datav)[endw],
+   main="S&P 500 Price-weighted Index and VTI") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", col="red") %>%
+   dySeries(name=colv[2], axis="y2", col="blue")
```



Writing Time Series To Files

The data from *Tiingo* is downloaded as `xts` time series, with a date-time index of class `POSIXct` (not `Date`).

The function `save()` writes objects to compressed binary `.RData` files.

The easiest way to share data between `R` and `Excel` is through `.csv` files.

The package *zoo* contains functions `write.zoo()` and `read.zoo()` for writing and reading *zoo* time series from `.txt` and `.csv` files.

The function `data.table::fread()` reads from `.csv` files over 6 times faster than the function `read.csv()`!

The function `data.table::fwrite()` writes to `.csv` files over 12 times faster than the function `write.csv()`, and 278 times faster than function `cat()`!

```
> # Save the environment to compressed .RData file
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/"
> save(sp500env, file=paste0(dirn, "sp500.RData"))
> # Save the ETF prices into CSV files
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/SP500/"
> for (symboln in ls(sp500env)) {
+   zoo::write.zoo(sp500env$symbol, file=paste0(dirn, symboln, ".csv
+ }  ##  end for
> # Or using lapply()
> filev <- lapply(ls(sp500env), function(symboln) {
+   xtsv <- get(symboln, envir=sp500env)
+   zoo::write.zoo(xtsv, file=paste0(dirn, symboln, ".csv"))
+   symboln
+ })  ##  end lapply
> unlist(filev)
> # Or using eapply() and data.table::fwrite()
> filev <- eapply(sp500env , function(xtsv) {
+   filen <- rutils::get_name(colnames(xtsv)[1])
+   data.table::fwrite(data.table::as.data.table(xtsv), file=paste0
+   filen
+ })  ##  end eapply
> unlist(filev)
```

Reading Time Series from Files

The function `load()` reads data from `.RData` files, and *invisibly* returns a vector of names of objects created in the workspace.

The function `Sys.glob()` listv files matching names obtained from wildcard expansion.

The easiest way to share data between `R` and `Excel` is through `.csv` files.

The function `as.Date()` parses `character` strings, and coerces `numeric` and `POSIXct` objects into `Date` objects.

The function `data.table::setDF()` coerces a *data table* object into a *data frame* using a *side effect*, without making copies of data.

The function `data.table::fread()` reads from `.csv` files over 6 times faster than the function `read.csv()`!

```
> # Load the environment from compressed .RData file
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/"
> load(file=paste0(dirn, "sp500.RData"))
> # Get all the .csv file names in the directory
> dirn <- "/Users/jerzy/Develop/lecture_slides/data/SP500/"
> filev <- Sys.glob(paste0(dirn, "*.csv"))
> # Create new environment for data
> sp500env <- new.env()
> for (filen in filev) {
+   xtsv <- xts::as.xts(zoo::read.csv.zoo(filen))
+   symboln <- rutils::get_name(colnames(xtsv)[1])
+   ##  symboln <- strsplit(colnames(xtsv), split="[.]")[[1]][1]
+   assign(symboln, xtsv, envir=sp500env)
+ }  ##  end for
> # Or using fread()
> for (filen in filev) {
+   xtsv <- data.table::fread(filen)
+   data.table::setDF(xtsv)
+   xtsv <- xts::xts(xtsv[, -1], as.Date(xtsv[, 1]))
+   symboln <- rutils::get_name(colnames(xtsv)[1])
+   assign(symboln, xtsv, envir=sp500env)
+ }  ##  end for
```

Downloading Stock Time Series From *Alpha Vantage*

Yahoo data quality deteriorated significantly in 2017, and *Google* data quality is also poor, leaving *Tiingo*, *Alpha Vantage*, and Quandl as the only major providers of free daily *OHLC* stock prices.

But Quandl doesn't provide free *ETF* prices, while *Alpha Vantage* does.

The function `getSymbols()` has a *method* for downloading time series data from *Alpha Vantage*, called `getSymbols.av()`.

Users must first obtain an *Alpha Vantage API key*, and then pass it in `getSymbols.av()` calls: https://www.alphavantage.co/

The function `adjustOHLC()` with argument `use.Adjusted=TRUE`, adjusts all the *OHLC* price columns, using the *Adjusted* price column.

```
> # Remove all files from environment(if necessary)
> rm(list=ls(sp500env), envir=sp500env)
> # Download in while loop from Alpha Vantage and copy into environm
> isdown <- symbolv %in% ls(sp500env)
> nattempts <- 0
> while ((sum(!isdown) > 0) & (nattempts < 10)) {
+   ##  Download data and copy it into environment
+   nattempts <- nattempts + 1
+   for (symboln in symbolv[!isdown]) {
+     cat("processing: ", symboln, "\n")
+     tryCatch(  ##  With error handler
+ quantmod::getSymbols(symboln, src="av", adjust=TRUE, auto.assign=T
+            output.size="full", api.key="T7JPW54ES8G75310"),
+ ##  error handler captures error condition
+ error=function(msg) {
+   print(paste0("Error handler: ", msg))
+ },  ##  end error handler
+ finally=print(paste0("Symbol = ", symboln))
+     )  ##  end tryCatch
+   }  ##  end for
+   ##  Update vector of symbols already downloaded
+   isdown <- symbolv %in% ls(sp500env)
+   Sys.sleep(2)  ##  Wait 2 seconds until next attempt
+ }  ##  end while
> # Adjust all OHLC prices in environment
> for (symboln in ls(sp500env)) {
+   assign(symboln,
+     adjustOHLC(get(x=symboln, envir=sp500env), use.Adjusted=TRUE),
+     envir=sp500env)
+ }  ##  end for
```

Downloading The *S&P500* Index Time Series From *Yahoo*

The *S&P500* stock market index is a capitalization-weighted average of the 500 largest U.S. companies, and covers about 80% of the U.S. stock market capitalization.

Notice: *Yahoo* no longer provides a public API for data.

There are workarounds but they're tedious.

Yahoo provides daily *OHLC* prices for the *S&P500* index (symbol *^GSPC*), and for the *S&P500* total return index (symbol *^SP500TR*).

But special characters in some stock symbols, like `"-"` or `"^"` are not allowed in `R` names.

For example, the symbol *^GSPC* for the *S&P500* stock market index isn't a valid name in `R`.

The function `setSymbolLookup()` creates valid names corresponding to stock symbols, which are then used by the function `getSymbols()` to create objects with the valid names.

Yahoo data quality deteriorated significantly in 2017, and *Google* data quality is also poor, leaving *Alpha Vantage* and Quandl as the only major providers of free daily *OHLC* stock prices.

```
> # Assign name SP500 to ^GSPC symbol
> quantmod::setSymbolLookup(SP500=list(name="^GSPC", src="yahoo"))
> quantmod::getSymbolLookup()
> # View and clear options
> options("getSymbols.sources")
> options(getSymbols.sources=NULL)
> # Download S&P500 prices into etfenv
> quantmod::getSymbols("SP500", env=etfenv,
+     adjust=TRUE, auto.assign=TRUE, from="1990-01-01")
>
> chart_Series(x=etfenv$SP500["2016/"],
+        TA="add_Vo()", name="S&P500 index")
```

Downloading The *DJIA* Index Time Series From *Yahoo*

The Dow Jones Industrial Average (*DJIA*) stock market index is a price-weighted average of the 30 largest U.S. companies (same number of shares per company).

Yahoo provides daily *OHLC* prices for the *DJIA* index (symbol *^DJI*), and for the *DJITR* total return index (symbol *DJITR*).

But special characters in some stock symbols, like `"-"` or `"^"` are not allowed in `R` names.

For example, the symbol *^DJI* for the *DJIA* stock market index isn't a valid name in `R`.

The function `setSymbolLookup()` creates valid names corresponding to stock symbols, which are then used by the function `getSymbols()` to create objects with the valid names.

```
> # Assign name DJIA to ^DJI symbol
> setSymbolLookup(DJIA=list(name="^DJI", src="yahoo"))
> getSymbolLookup()
> # view and clear options
> options("getSymbols.sources")
> options(getSymbols.sources=NULL)
> # Download DJIA prices into etfenv
> quantmod::getSymbols("DJIA", env=etfenv,
+     adjust=TRUE, auto.assign=TRUE, from="1990-01-01")
> chart_Series(x=etfenv$DJIA["2016/"],
+        TA="add_Vo()", name="DJIA index")
```

Calculating Prices and Returns From *OHLC* Data

The function `na.locf()` from package *zoo* replaces `NA` values with the most recent non-`NA` values prior to it.

The function `na.locf()` with argument `fromLast=TRUE` replaces `NA` values with non-`NA` values in reverse order, starting from the end.

The function `rutils::get_name()` extracts symbol names (tickers) from a vector of character strings.

```
> pricev <- eapply(sp500env, quantmod::Cl)
> pricev <- rutils::do_call(cbind, pricev)
> # Carry forward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> # Get first column name
> colnames(pricev[, 1])
> rutils::get_name(colnames(pricev[, 1]))
> # Modify column names
> colnames(pricev) <- rutils::get_name(colnames(pricev))
> # Or
> # colnames(pricev) <- do.call(rbind,
> #   strsplit(colnames(pricev), split="[.]"))[, 1]
> # Calculate percentage returns
> retp <- xts::diff.xts(pricev)/rutils::lagit(pricev, pad_zeros=FALS
> # Select a random sample of 100 prices and returns
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> samplev <- sample(NCOL(retp), s=100, replace=FALSE)
> prices100 <- pricev[, samplev]
> returns100 <- retp[, samplev]
> # Save the data into binary files
> save(pricev, prices100,
+      file="/Users/jerzy/Develop/lecture_slides/data/sp500_prices.
> save(retp, returns100,
+      file="/Users/jerzy/Develop/lecture_slides/data/sp500_returns
```

Downloading Stock Prices From Polygon

Polygon is a premium provider of live and historical stock price data, both daily and intraday (minutes).

Polygon provides 2 years of daily historical stock prices for free. But users must first obtain a Polygon *API key*.

Polygon provides the historical *OHLC* stock prices in *JSON* format.

JSON (JavaScript Object Notation) is a data format consisting of symbol-value pairs.

The package *jsonlite* contains functions for managing data in *JSON* format.

The functions `fromJSON()` and `toJSON()` convert data from *JSON* format to `R` objects, and vice versa.

The functions `read_json()` and `write_json()` read and write *JSON* format data in files.

The function `download.file()` downloads data from an internet website `URL` and writes it to a file.

```
> # Setup code
> symboln <- "SPY"
> startd <- as.Date("1990-01-01")
> todayd <- Sys.Date()
> tspan <- "day"
> # Replace below your own Polygon API key
> apikey <- "SEpnsBpiRyONMJdl48r6dOo0_pjmCu5r"
> # Create url for download
> urll <- paste0("https://api.polygon.io/v2/aggs/ticker/", symboln,
> # Download SPY OHLC prices in JSON format from Polygon
> ohlc <- jsonlite::read_json(urll)
> class(ohlc)
> NROW(ohlc)
> names(ohlc)
> # Extract list of prices from json object
> ohlc <- ohlc$results
> # Coerce from list to matrix
> ohlc <- lapply(ohlc, unlist)
> ohlc <- do.call(rbind, ohlc)
> # Coerce time from milliseconds to dates
> datev <- ohlc[, "t"]/1e3
> datev <- as.POSIXct(datev, origin="1970-01-01")
> datev <- as.Date(datev)
> tail(datev)
> # Coerce from matrix to xts
> ohlc <- ohlc[, c("o","h","l","c","v","vw")]
> colnames(ohlc) <- c("Open", "High", "Low", "Close", "Volume", "VW
> ohlc <- xts::xts(ohlc, order.by=datev)
> tail(ohlc)
> # Save the xts time series to compressed RData file
> save(ohlc, file="/Users/jerzy/Data/spy_daily.RData")
> # Candlestick plot of SPY OHLC prices
> dygraphs::dygraph(ohlc[, 1:4], main=paste("Candlestick Plot of", s
+   dygraphs::dyCandlestick()
```

Downloading Multiple Stock Prices From Polygon

The stock prices for multiple stocks can be downloaded in a `while()` loop.

```
> # Select ETF symbols for asset allocation
> symbolv <- c("SPY", "VTI", "QQQ", "VEU", "EEM", "XL
+ "XLE", "XLF", "XLV", "XLI", "XLB", "XLK", "XLU", "V
+ "IWB", "IWD", "IWF", "IEF", "TLT", "VNQ", "DBC", "G
+ "VXX", "SVXY", "MTUM", "IVE", "VLUE", "QUAL", "VTV"
> # Setup code
> etfenv <- new.env()  ##  New environment for data
> # Boolean vector of symbols already downloaded
> isdown <- symbolv %in% ls(etfenv)
```

```
> # Download data from Polygon using while loop
> while (sum(!isdown) > 0) {
+   for (symboln in symbolv[!isdown]) {
+     cat("Processing:", symboln, "\n")
+     tryCatch({  ##  With error handler
+ ##  Download OHLC bars from Polygon into JSON format file
+ urll <- paste0("https://api.polygon.io/v2/aggs/ticker/", symboln, "/range/1/",
+ ohlc <- jsonlite::read_json(urll)
+ ##  Extract list of prices from json object
+ ohlc <- ohlc$results
+ ##  Coerce from list to matrix
+ ohlc <- lapply(ohlc, unlist)
+ ohlc <- do.call(rbind, ohlc)
+ ##  Coerce time from milliseconds to dates
+ datev <- ohlc[, "t"]/1e3
+ datev <- as.POSIXct(datev, origin="1970-01-01")
+ datev <- as.Date(datev)
+ ##  Coerce from matrix to xts
+ ohlc <- ohlc[, c("o","h","l","c","v","vw")]
+ colnames(ohlc) <- paste0(symboln, ".", c("Open", "High", "Low", "Close", "Volu
+ ohlc <- xts::xts(ohlc, order.by=datev)
+ ##  Save to environment
+ assign(symboln, ohlc, envir=etfenv)
+ Sys.sleep(1)
+ },
+     error={function(msg) print(paste0("Error handler: ", msg))},
+     finally=print(paste0("Symbol = ", symboln))
+     )  ##  end tryCatch
+   }  ##  end for
+   ##  Update vector of symbols already downloaded
+   isdown <- symbolv %in% ls(etfenv)
+ }  ##  end while
> save(etfenv, file="/Users/jerzy/Develop/lecture_slides/data/etf_data.RData")
```

Calculating the Stock Alphas, Betas, and Other Performance Statistics

The package *PerformanceAnalytics* contains functions for calculating risk and performance statistics, such as the *variance*, *skewness*, *kurtosis*, *beta*, *alpha*, etc.

The function `PerformanceAnalytics::table.CAPM()` calculates the *beta* β and *alpha* α values, the *Treynor* ratio, and other performance statistics.

The function `PerformanceAnalytics::table.Stats()` calculates a data frame of risk and return statistics of the return distributions.

```
> pricev <- eapply(etfenv, quantmod::Cl)
> pricev <- do.call(cbind, pricev)
> # Drop ".Close" from colnames
> colnames(pricev) <- do.call(rbind, strsplit(colnames(pricev), spli
> # Calculate the log returns
> retp <- xts::diff.xts(log(pricev))
> # Copy prices and returns into etfenv
> etfenv$pricev <- pricev
> etfenv$retp <- retp
> # Copy symbolv into etfenv
> etfenv$symbolv <- symbolv
> # Calculate the risk-return statistics
> riskstats <- PerformanceAnalytics::table.Stats(retp)
> # Transpose the data frame
> riskstats <- as.data.frame(t(riskstats))
> # Add Name column
> riskstats$Name <- rownames(riskstats)
> # Copy riskstats into etfenv
> etfenv$riskstats <- riskstats
> # Calculate the beta, alpha, Treynor ratio, and other performance
> capmstats <- PerformanceAnalytics::table.CAPM(Ra=retp[, symbolv],
+                                               Rb=retp[, "VTI"], scale=2
> colv <- strsplit(colnames(capmstats), split=" ")
> colv <- do.call(cbind, colv)[1, ]
> colnames(capmstats) <- colv
> capmstats <- t(capmstats)
> capmstats <- capmstats[, -1]
> colv <- colnames(capmstats)
> whichv <- match(c("Annualized Alpha", "Information Ratio", "Treyno
> colv[whichv] <- c("Alpha", "Information", "Treynor")
> colnames(capmstats) <- colv
> capmstats <- capmstats[order(capmstats[, "Alpha"], decreasing=TRUE
> # Copy capmstats into etfenv
> etfenv$capmstats <- capmstats
> save(etfenv, file="/Users/jerzy/Develop/lecture_slides/data/etf_da
```

Scraping *S&P500* Stock Index Constituents From Websites

The *S&P500* index constituents change over time, and *Standard & Poor's* replaces companies that have decreased in capitalization with ones that have increased.

The *S&P500* index may contain more than 500 stocks because some companies have several share classes of stock.

The *S&P500* index constituents may be scraped from websites like Wikipedia, using dedicated packages.

The function `getURL()` from package *RCurl* downloads the *html* text data from an internet website `URL`.

The function `readHTMLTable()` from package *XML* extracts tables from *html* text data or from a remote `URL`, and returns them as a list of *data frames* or matrices.

`readHTMLTable()` can't parse secure `URLs`, so they must first be downloaded using function `getURL()`, and then parsed using `readHTMLTable()`.

```
> library(RCurl)  ##  Load package RCurl
> library(XML)  ##  Load package XML
> # Download text data from URL
> sp500 <- getURL(
+   "https://en.wikipedia.org/wiki/List_of_S%26P500_companies")
> # Extract tables from the text data
> sp500 <- readHTMLTable(sp500)
> str(sp500)
> # Extract colnames of data frames
> lapply(sp500, colnames)
> # Extract S&P500 constituents
> sp500 <- sp500[[1]]
> head(sp500)
> # Create valid R names from symbols containing "-" or "."characte
> sp500$namev <- gsub("-", "_", sp500$Ticker)
> sp500$namev <- gsub("[.]", "_", sp500$names)
> # Write data frame of S&P500 constituents to CSV file
> write.csv(sp500,
+   file="/Users/jerzy/Develop/lecture_slides/data/sp500_Yahoo.csv"
+   row.names=FALSE)
```

Downloading *S&P500* Time Series Data From *Yahoo*

Before time series data for the *S&P500* index constituents can be downloaded from *Yahoo*, it's necessary to create valid names corresponding to symbols containing special characters like "-".

The function `setSymbolLookup()` creates a lookup table for *Yahoo* symbols, using valid names in `R`.

For example *Yahoo* uses the symbol "BRK-B", which isn't a valid name in `R`, but can be mapped to "BRK_B", using the function `setSymbolLookup()`.

```
> library(rutils)  ##  Load package rutils
> # Load data frame of S&P500 constituents from CSV file
> sp500 <- read.csv(file="/Users/jerzy/Develop/lecture_slides/data/s
> # Register symbols corresponding to R names
> for (indeks in 1:NROW(sp500)) {
+   cat("processing: ", sp500$Ticker[indeks], "\n")
+   setSymbolLookup(structure(
+     list(list(name=sp500$Ticker[indeks])),
+     names=sp500$names[indeks]))
+ }  ##  end for
> sp500env <- new.env()  ##  new environment for data
> # Remove all files (if necessary)
> rm(list=ls(sp500env), envir=sp500env)
> # Download data and copy it into environment
> rutils::get_data(sp500$names,
+    env_out=sp500env, startd="1990-01-01")
> # Or download in loop
> for (symboln in sp500$names) {
+   cat("processing: ", symboln, "\n")
+   rutils::get_data(symboln,
+    env_out=sp500env, startd="1990-01-01")
+ }  ##  end for
> save(sp500env, file="/Users/jerzy/Develop/lecture_slides/data/sp5
> chart_Series(x=sp500env$BRKB["2016/"],
+        TA="add_Vo()", name="BRK-B stock")
```

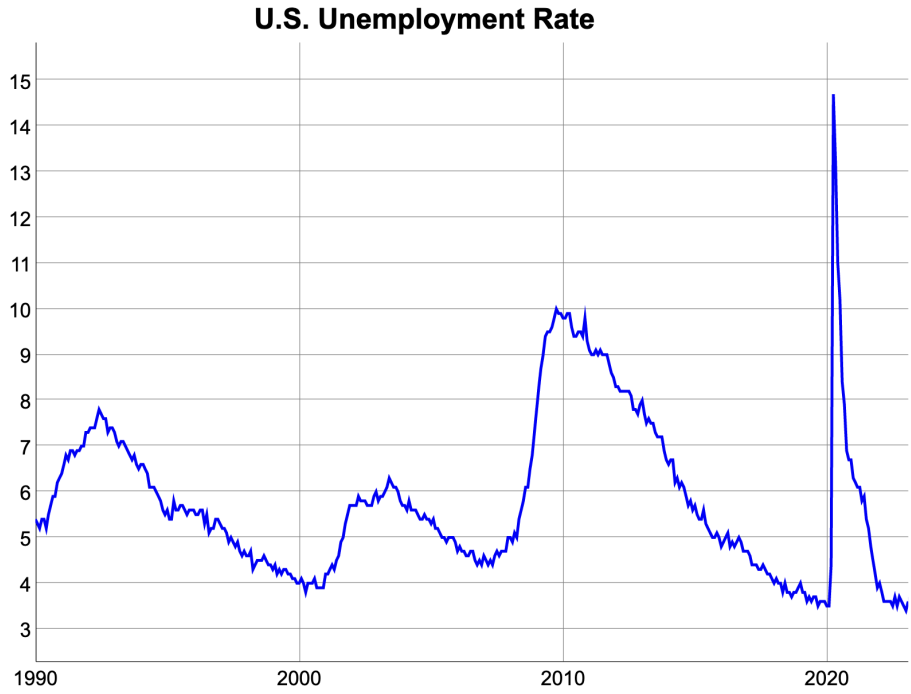
Downloading *FRED* Time Series Data

FRED is a database of economic time series maintained by the Federal Reserve Bank of St. Louis:
http://research.stlouisfed.org/fred2/

The function `getSymbols()` downloads time series data into the specified *environment*.

`getSymbols()` can download *FRED* data with the argument `"src"` set to `FRED`.

If the argument `"auto.assign"` is set to `FALSE`, then `getSymbols()` returns the data, instead of assigning it silently.




```
> # Download U.S. unemployment rate data
> unrate <- quantmod::getSymbols("UNRATE",
+    auto.assign=FALSE, src="FRED")
> # Plot U.S. unemployment rate data
> dygraphs::dygraph(unrate["1990/"], main="U.S. Unemployment Rate")
+   dyOptions(colors="blue", strokeWidth=2)
> # Or
> quantmod::chart_Series(unrate["1990/"], name="U.S. Unemployment Ra
```

The *Quandl* Database

Quandl is a distributor of third party data, and offers several million financial, economic, and social datasets.

Much of the Quandl data is free, while premium data can be obtained under a temporary license.

Quandl provides online help and a guide to its datasets:

- https://www.quandl.com/help/r
- https://www.quandl.com/browse
- https://www.quandl.com/blog/getting-started-with-the-quandl-api
- https://www.quandl.com/blog/stock-market-data-guide

Quandl provides stock prices, stock fundamentals, financial ratios, zoo::indexes, options and volatility, earnings estimates, analyst ratings, etc.:

- https://www.quandl.com/blog/api-for-stock-data

```
> install.packages("devtools")
> library(devtools)
> # Install package Quandl from github
> install_github("quandl/R-package")
> library(Quandl)  ##  Load package Quandl
> # Register Quandl API key
> Quandl.api_key("pVJi9Nv3V8CD3Js5s7Qx")
> # Get short description
> packageDescription("Quandl")
> # Load help page
> help(package="Quandl")
> # Remove Quandl from search path
> detach("package:Quandl")
```

Quandl has developed an R package called *Quandl* that allows downloading data from Quandl directly into R.

To make more than 50 downloads a day, you need to register your *Quandl API key* using the function `Quandl.api_key()`,

Downloading Time Series Data from *Quandl*

Quandl data can be downloaded directly into R using the function `Quandl()`.

The dots `"..."` argument of the `Quandl()` function accepts additional parameters to the *Quandl API*,

Quandl datasets have a unique *Quandl code* in the format `"database/ticker"`, which can be found on the Quandl website for that dataset:
https://www.quandl.com/data/WIKI?keyword=aapl

WIKI is a user maintained free database of daily prices for 3,000 U.S. stocks,
https://www.quandl.com/data/WIKI

SEC is a free database of stock fundamentals extracted from *SEC 10Q* and *10K* filings (but not harmonized),
https://www.quandl.com/data/SEC

RAYMOND is a free database of harmonized stock fundamentals, based on the *SEC* database,
https://www.quandl.com/data/RAYMOND-Raymond https://www.quandl.com/data/RAYMOND-Raymond?keyword=aapl

```
> library(rutils)  ##  Load package rutils
> # Download EOD AAPL prices from WIKI free database
> pricev <- Quandl(code="WIKI/AAPL",
+   type="xts", startd="1990-01-01")
> x11(width=14, height=7)
> chart_Series(pricev["2016", 1:4], name="AAPL OHLC prices")
> # Add trade volume in extra panel
> add_TA(pricev["2016", 5])
> # Download euro currency rates
> pricev <- Quandl(code="BNP/USDEUR",
+     startd="2013-01-01",
+     endd="2013-12-01", type="xts")
> # Download multiple time series
> pricev <- Quandl(code=c("NSE/OIL", "WIKI/AAPL"),
+     startd="2013-01-01", type="xts")
> # Download AAPL gross profits
> prof_it <- Quandl("RAYMOND/AAPL_GROSS_PROFIT_Q", type="xts")
> chart_Series(prof_it, name="AAPL gross profits")
> # Download Hurst time series
> pricev <- Quandl(code="PE/AAPL_HURST",
+     startd="2013-01-01", type="xts")
> chart_Series(pricev["2016/", 1], name="AAPL Hurst")
```

Stock Index and Instrument Metadata on *Quandl*

Instrument metadata specifies properties of instruments, like its currency, contract size, tick value, delivery months, start date, etc.

Quandl provides instrument metadata for stock indices, futures, and currencies:

https://www.quandl.com/blog/useful-listv

Quandl also provides constituents for stock indices, for example the *S&P500*, *Dow Jones Industrial Average*, *NASDAQ Composite*, *FTSE 100*, etc.

```
> # Load S&P500 stock Quandl codes
> sp500 <- read.csv(
+   file="/Users/jerzy/Develop/lecture_slides/data/sp500_quandl.csv"
> # Replace "-" with "_" in symbols
> sp500$free_code <- gsub("-", "_", sp500$free_code)
> head(sp500)
> # vector of symbols in sp500 frame
> tickers <- gsub("-", "_", sp500$ticker)
> # Or
> tickers <- matrix(unlist(
+   strsplit(sp500$free_code, split="/"),
+   use.names=FALSE), ncol=2, byrow=TRUE)[, 2]
> # Or
> tickers <- do_call_rbind(
+   strsplit(sp500$free_code, split="/"))[, 2]
```

Downloading Multiple Time Series from *Quandl*

Time series data for a portfolio of stocks can be downloaded by performing a loop over the function `Quandl()` from package Quandl.

The `assign()` function assigns a value to an object in a specified *environment*, by referencing it using a character string (name).

```
> sp500env <- new.env()  ##  new environment for data
> # Remove all files (if necessary)
> rm(list=ls(sp500env), envir=sp500env)
> # Boolean vector of symbols already downloaded
> isdown <- tickers %in% ls(sp500env)
> # Download data and copy it into environment
> for (ticker in tickers[!isdown]) {
+   cat("processing: ", ticker, "\n")
+   datav <- Quandl(code=paste0("WIKI/", ticker),
+              startd="1990-01-01", type="xts")[, -(1:7)]
+   colnames(datav) <- paste(ticker,
+     c("Open", "High", "Low", "Close", "Volume"), sep=".")
+   assign(ticker, datav, envir=sp500env)
+ }  ##  end for
> save(sp500env, file="/Users/jerzy/Develop/lecture_slides/data/sp5
> chart_Series(x=sp500env$XOM["2016/"], TA="add_Vo()", name="XOM st
```

Downloading Futures Time Series from *Quandl*

Quandl provides the Wiki CHRIS Database of time series of prices for 600 different futures contracts.

The Wiki CHRIS Database contains daily *OHLC* prices for continuous futures contracts.

A continuous futures contract is a time series of prices obtained by chaining together prices from consecutive futures contracts.

The data is curated by the Quandl community from data provided by the *CME*, *ICE*, *LIFFE*, and other exchanges.

The *Quandl codes* are specified as `CHRIS/{EXCHANGE}_{CODE}{DEPTH}`, where `{DEPTH}` is the depth of the chained contract.

The chained front month contracts have depth 1, the back month contracts have depth 2, etc.

The continuous front and back month contracts allow building continuous futures curves.

Quandl data can be downloaded directly into `R` using the function `Quandl()`.

```
> library(Quandl)
> # Register Quandl API key
> Quandl.api_key("pVJi9Nv3V8CD3Js5s7Qx")
> # Download E-mini S&P500 futures prices
> pricev <- Quandl(code="CHRIS/CME_ES1",
+   type="xts", startd="1990-01-01")
> pricev <- pricev[, c("Open", "High", "Low", "Last", "Volume")]
> colnames(pricev)[4] <- "Close"
> # Plot the prices
> x11(width=5, height=4)  ##  Open x11 for plotting
> chart_Series(x=pricev["2008-06/2009-06"],
+        TA="add_Vo()", name="S&P500 Futures")
> # Plot dygraph
> dygraphs::dygraph(pricev["2008-06/2009-06", -5],
+   main="S&P500 Futures") %>%
+   dyCandlestick()
```

For example, the *Quandl code* for the continuous *E-mini S&P500* front month futures is `CHRIS/CME_ES1`, while for the back month it's `CHRIS/CME_ES2`, for the second back month it's `CHRIS/CME_ES3`, etc.

The *Quandl code* for the *E-mini Oil* futures is `CHRIS/CME_QM1`, for the *E-mini euro FX* futures is `CHRIS/CME_E71`, etc.

Downloading *VIX* Futures Files from CBOE

The CFE (CBOE Futures Exchange) provides daily CBOE Historical Data for Volatility Futures, including the *VIX* futures.

The CBOE data incudes *OHLC* prices and also the *settlement* price (in column `"Settle"`).

The *settlement* price is usually defined as the weighted average price (*WAP*) or the midpoint price, and is different from the *Close* price.

The *settlement* price is used for calculating the daily *mark to market* (value) of the futures contract.

Futures exchanges require that counterparties exchange (settle) the *mark to market* value of the futures contract daily, to reduce counterparty default risk.

The function `download.file()` downloads files from the internet.

The function `tryCatch()` executes functions and expressions, and handles any *exception conditions* produced when they are evaluated.

```
> # Read CBOE futures expiration dates
> datev <- read.csv(file="/Users/jerzy/Develop/lecture_slides/data/f
+   row.names=1)
> # Create directory for data
> dirn <- "/Users/jerzy/Develop/data/vix_data"
> dir.create(dirn)
> namev <- rownames(datev)
> filev <- file.path(dirn, paste0(namev, ".csv"))
> filelog <- file.path(dirn, "log_file.txt")
> urlcboe <- "https://markets.cboe.com/us/futures/market_statistics/
> urlv <- paste0(urlcboe, datev[, 1])
> # Download files in loop
> for (it in seq_along(urlv)) {
+     tryCatch(  ##  Warning and error handler
+   download.file(urlv[it], destfile=filev[it], quiet=TRUE),
+ ##  Warning handler captures warning condition
+ warning=function(msg) {
+   cat(paste0("Warning handler: ", msg, "\n"), file=filelog, appen
+ },  ##  end warning handler
+ ##  Error handler captures error condition
+ error=function(msg) {
+   cat(paste0("Error handler: ", msg, "\n"), append=TRUE)
+ },  ##  end error handler
+ finally=cat(paste0("Processing file name = ", filev[it], "\n"), ap
+     )  ##  end tryCatch
+ }  ##  end for
```

Downloading *VIX* Futures Data Into an Environment

The function `quantmod::getSymbols()` with the parameter `src="cfe"` downloads CFE data into the specified *environment*. (But this requires first loading the package *qmao*.)

Currently `quantmod::getSymbols()` doesn't download the most recent data.

```
> # Create new environment for data
> vixenv <- new.env()
> # Download VIX data for the months 6, 7, and 8 in 2018
> library(qmao)
> quantmod::getSymbols("VX", Months=1:12,
+   Years=2018, src="cfe", auto.assign=TRUE, env=vixenv)
> # Or
> qmao::getSymbols.cfe(Symbols="VX",
+   Months=6:8, Years=2018, env=vixenv,
+   verbose=FALSE, auto.assign=TRUE)
> # Calculate the classes of all the objects
> # In the environment vixenv
> unlist(eapply(vixenv, function(x) {class(x)[1]}))
> class(vixenv$VX_M18)
> colnames(vixenv$VX_M18)
> # Save the data to a binary file called "vix_cboe.RData".
> save(vixenv,
+   file="/Users/jerzy/Develop/data/vix_data/vix_cboe.RData")
```

Homework Assignment

No homework!